

ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**Nghiên Cứu Và Triển Khai Mô Hình Nhận
Điện Từ Khóa Tiếng Nói Trên Thiết Bị Nhúng
Sử Dụng Học Sâu**

Hệ thống nhúng và IoT

Sinh viên: **NGUYỄN TRẦN HẠO MINH**
MSSV: 22139042

Hướng dẫn: PGS.TS. **VÕ MINH HUÂN**

TP. HỒ CHÍ MINH, 06/2026

ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**Nghiên Cứu Và Triển Khai Mô Hình Nhận
Điện Từ Khóa Tiếng Nói Trên Thiết Bị Nhúng
Sử Dụng Học Sâu**

Hệ thống nhúng và IoT

Sinh viên: **NGUYỄN TRẦN HẠO MINH**

MSSV: 22139042

Hướng dẫn: PGS.TS. **VÕ MINH HUÂN**

TP. HỒ CHÍ MINH, 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Nghiên Cứu Và Triển Khai Mô Hình Nhận Diện Từ Khoá Tiếng Nói Trên Thiết Bị Nhúng Sử Dụng Học Sâu

Sinh viên: Nguyễn Trần Hạo Minh

MSSV: 22139042

Hướng dẫn: PSG.TS. Võ Minh Huân

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Hướng giải quyết của đề tài thể hiện tính mới mẻ và tiềm năng ứng dụng thực tiễn

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Lựa chọn phương pháp tiếp cận có cơ sở lý luận vững chắc, đảm bảo độ tin cậy cao

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Kết quả thu được tương thích với mục tiêu ban đầu

4. Kết luận và đề xuất:

Tổng kết vấn đề xúc tích, phản hồi chính xác cách đặt vấn đề ở phần mở đầu

5. Hình thức trình bày và bố cục báo cáo:

Hình thức báo cáo chuẩn mực, bố cục chặt chẽ và văn phong khoa học đồng nhất. Việc trình bày trau chuốt cho thấy tuân thủ nghiêm ngặt định dạng mẫu cho thấy năng lực nghiên cứu của sinh viên

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện phong thái làm việc tích cực cùng kỹ năng trao đổi, giao tiếp, hiệu quả trong quá trình thực hiện khóa luận

7. Tài liệu trích dẫn

Đảm bảo tính trung thực học thuật; các nguồn tài liệu tham khảo được trích dẫn đầy đủ

8. Đánh giá về sự trùng lặp của đề tài

Tỉ lệ tương đồng của đề tài nằm trong ngưỡng quy định cho phép

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Không có nhược điểm và thiếu sót nào cần hiệu chỉnh

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Sinh viên duy trì tinh thần trách nhiệm cao

Đề nghị của giảng viên hướng dẫn

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp

Tp. HCM, ngày 24 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)



PGS. TS. Võ Minh Huân

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, tôi đã nhận được sự giúp đỡ và hỗ trợ nhiệt tình từ nhiều cá nhân và tập thể. Tôi xin gửi lời cảm ơn chân thành tới thầy Võ Minh Huân, người đã trực tiếp hướng dẫn và chỉ bảo tận tình cho tôi trong suốt thời gian qua. Tôi cũng xin cảm ơn quý Thầy/Cô trường Đại học Công Nghệ Kỹ Thuật đã giảng dạy và trang bị kiến thức cho tôi trong suốt khóa học. Cảm ơn gia đình và bạn bè đã luôn động viên, khích lệ tinh thần để tôi hoàn thành tốt nhiệm vụ của mình. Do kiến thức và kinh nghiệm còn hạn chế, đồ án không tránh khỏi những thiếu sót. Tôi rất mong nhận được sự đóng góp ý kiến của quý Thầy/Cô và các bạn. Xin trân trọng cảm ơn!

Nhóm thực hiện đồ án tốt nghiệp
(Ký và ghi rõ họ tên)

A handwritten signature in blue ink, appearing to read 'Nguyễn Trần Hạo Minh', is written on a light blue grid background.

Nguyễn Trần Hạo Minh

LỜI CAM ĐOAN

Nhóm thực hiện đồ án tốt nghiệp cam đoan đề tài thực hiện dựa vào một số tài liệu trước đó và không sao chép nội dung, kết quả của đồ án khác. Các nội dung tham khảo đã được trích dẫn đầy đủ.

Nhóm thực hiện đồ án tốt nghiệp
(Ký và ghi rõ họ tên)

A handwritten signature in blue ink, consisting of several fluid, connected strokes, positioned on a light-colored rectangular background.

Nguyễn Trần Hạo Minh

TÓM TẮT NỘI DUNG

Nhận dạng từ khóa (Keyword Spotting – KWS) là thành phần nền tảng trong các hệ thống điều khiển bằng giọng nói cho thiết bị IoT, đóng vai trò luôn bật để phát hiện lệnh kích hoạt từ người dùng. Tuy nhiên, việc triển khai các mô hình học sâu hiệu quả lên thiết bị biên đặt ra ba thách thức đồng thời: giới hạn bộ nhớ, ràng buộc điện năng và nghịch lý Power Paradox – hiện tượng mô hình tối ưu FLOPs quá mức lại tiêu thụ điện nhiều hơn trên phần cứng thực tế do tần suất truy xuất bộ nhớ dày đặc.

Nghiên cứu này đề xuất QANet-Lite, một kiến trúc mạng nơ-ron tích chập siêu nhẹ được thiết kế chuyên biệt cho bài toán KWS trên thiết bị biên nhằm giải quyết triệt để các thách thức trên. Kiến trúc trung tâm là khối QuadAxialBlock, trích xuất đồng thời bốn loại đặc trưng bổ sung nhau từ cùng một feature map thông qua bốn nhánh song song: nhánh Local (tích chập tách biệt chiều sâu 3×3) nắm bắt tương quan không gian cục bộ hai chiều; nhánh Time (tích chập 1×3) học đặc trưng theo trục thời gian; nhánh Freq (tích chập 3×1) học cấu trúc hài hòa theo trục tần số; và nhánh Channel (Channel Attention Module – CAM kết hợp tích chập 1×1) tái cân bằng tầm quan trọng của từng kênh đặc trưng theo ngữ cảnh toàn cục. Bốn nhánh được kết hợp thông qua bộ định tuyến động (dynamic router) học trọng số thích nghi theo nội dung đầu vào, thay vì tổng hợp cố định. Cơ chế CAM giúp cải thiện mạnh mẽ độ chính xác phân loại bằng cách ức chế nhiễu nền, trong khi cấu trúc đường ống xử lý phân nhánh tổng thể của QuadAxialBlock giúp cân bằng tỷ lệ tính toán và bộ nhớ (Arithmetic Intensity), tránh hiện tượng vắt kiệt năng lượng phần cứng.

Để nâng cao độ chính xác mà không làm tăng chi phí suy luận, nghiên cứu áp dụng kỹ thuật Chưng cất Tri thức (Knowledge Distillation) liên kiến trúc từ mô hình Giáo viên Audio Spectrogram Transformer (AST, 85,4 triệu tham số, độ chính xác 98,17%) sang mô hình Học sinh QANet-Lite. Mô hình được huấn luyện trên tập dữ liệu Google Speech Commands V2 gồm 105.829 mẫu âm thanh thuộc 35 lớp từ khóa, với hàm mất mát tổng hợp kết hợp Cross-Entropy cứng và KL Divergence mềm ở nhiệt độ $T = 4$.

Mô hình sau huấn luyện được chuyển đổi sang định dạng ONNX và triển khai lên thiết bị NVIDIA Jetson Nano Developer Kit để đánh giá trong điều kiện thời gian thực.

Kết quả thực nghiệm cho thấy QANet-Lite đạt độ chính xác 95,62% với kích thước mô hình chỉ 0,64 MB và độ trễ suy luận 23,84 ms ($RTF = 0,024$), đáp ứng hoàn toàn ràng buộc thời gian thực. Đặc biệt, nghiên cứu cung cấp bằng chứng định lượng xác nhận hiện tượng Power Paradox: MobileNetV2 có FLOPs thấp nhất (0,20 G) nhưng tiêu thụ điện cao nhất (7,98 W) do thông lượng quá cao gây nghẽn cổ chai bộ nhớ. Ngược lại, QANet-Lite duy trì mức tiêu thụ điện tối ưu ở mức 6,28 W nhờ cấu trúc song song đặc thù giúp điều tiết nhịp độ tính toán đường ống, đạt hiệu quả năng lượng 6,68 IPS/W – cân bằng vượt trội so với các baseline. Kết quả Ablation Study xác nhận đóng góp độc lập của cả cơ chế CAM lẫn Chứng cất Tri thức vào hiệu năng cuối cùng.

Từ khóa: Nhận dạng từ khóa, học sâu nhẹ, thiết bị biên, chứng cất tri thức, cơ chế chú ý kênh, Log-Mel Spectrogram, NVIDIA Jetson Nano.

Mục lục

1	MỞ ĐẦU	1
1.1	Giới thiệu	1
1.2	Mục tiêu đề tài	2
1.3	Đối tượng và phạm vi nghiên cứu	3
1.3.1	Đối tượng nghiên cứu	3
1.3.2	Phạm vi nghiên cứu	3
1.4	Phương pháp nghiên cứu	4
1.5	Bố cục của đề tài	4
2	CƠ SỞ LÝ THUYẾT	6
2.1	Tổng quan về Học sâu (Deep Learning)	6
2.1.1	Mạng nơ-ron nhân tạo (Artificial Neural Networks)	6
2.1.2	Quá trình huấn luyện và tối ưu hoá	8
2.2	Mạng nơ-ron tích chập (Convolutional Neural Networks)	11
2.2.1	Nguyên lý tích chập và chia sẻ trọng số	11
2.2.2	Các thành phần cơ bản của CNN	13
2.3	Xử lý tín hiệu âm thanh	14
2.3.1	Biểu diễn tín hiệu âm thanh số	14
2.3.2	Biến đổi Fourier thời gian ngắn (STFT)	15
2.3.3	Log-Mel Spectrogram	16
2.3.4	Tăng cường dữ liệu âm thanh	17
2.4	Kiến trúc CNN nhẹ cho thiết bị nhúng	18
2.4.1	Động lực thiết kế mạng nhẹ	18

2.4.2	Tích chập tách biệt chiều sâu (Depthwise Separable Convolution)	18
2.4.3	Khối Inverted Residual (MobileNetV2)	20
2.4.4	Nghịch lý hiệu quả năng lượng (Power Paradox)	20
2.5	Cơ chế chú ý (Attention Mechanism)	21
2.5.1	Động lực và phân loại	21
2.5.2	Squeeze-and-Excitation (SE)	22
2.5.3	CBAM (Convolutional Block Attention Module)	23
2.6	Nhận dạng từ khóa (Keyword Spotting)	25
2.6.1	Định nghĩa và đặc tả bài toán	25
2.6.2	Tập dữ liệu Google Speech Commands V2	26
2.6.3	Thách thức triển khai trên thiết bị biên	26
2.6.4	Các kiến trúc KWS tiêu biểu	27
2.7	Chưng cất tri thức (Knowledge Distillation)	28
2.7.1	Kiến trúc Giáo viên – Học sinh	28
2.7.2	Hàm mất mát trong Knowledge Distillation	29
2.7.3	Chưng cất liên kiến trúc (Cross-Architecture KD)	29
2.8	Các phương pháp và thang đo đánh giá	30
2.8.1	Đánh giá độ chính xác phân loại	30
2.8.2	Đánh giá độ phức tạp và hiệu năng phần cứng	31
3	THIẾT KẾ HỆ THỐNG	32
3.1	Tổng quan hệ thống	32
3.1.1	Kiến trúc tổng thể	32
3.1.2	Tiền xử lý đầu vào	34
3.2	Mô hình Giáo viên (Teacher Model)	35
3.2.1	Audio Spectrogram Transformer	35
3.2.2	Vai trò và giới hạn triển khai	36
3.3	Kiến trúc mô hình học sinh đề xuất (QANet-Lite)	37
3.3.1	Tổng quan thiết kế	37
3.3.2	Mô-đun chú ý kênh (Channel Attention Module – CAM)	39
3.3.3	Khối trích xuất đặc trưng đa nhánh (QuadAxialBlock)	41

3.3.4	Khối giảm mẫu lai (CrossAxisDownsample)	44
3.3.5	Đầu phân loại (Classification Head)	46
3.3.6	Phân tích tổng hợp và vị trí trong hệ sinh thái KWS	47
3.4	Chiến lược chứng cất tri thức	48
3.4.1	Tổng quan luồng huấn luyện	48
3.4.2	Hàm mất mát tổng hợp	48
3.4.3	Lựa chọn và lý giải siêu tham số α và T	49
3.5	Triển khai lên phần cứng	50
3.5.1	Chuyển đổi sang định dạng ONNX	50
3.5.2	Quy trình đánh giá hiệu năng suy luận (Benchmark Pipeline) . .	50
4	THỰC NGHIỆM VÀ ĐÁNH GIÁ	52
4.1	Môi trường thực nghiệm	52
4.1.1	Tập dữ liệu	52
4.1.2	Cấu hình phần cứng	53
4.1.3	Cấu hình huấn luyện	53
4.2	Đánh giá trên máy trạm	54
4.2.1	So sánh độ chính xác với các mô hình baseline	54
4.3	Đánh giá trên phần cứng biên (Jetson Nano)	55
4.3.1	Tốc độ xử lý và Độ trễ (Latency & Throughput)	55
4.3.2	Tiêu thụ điện năng và Nghịch lý Power Paradox	55
4.3.3	Sự đánh đổi (Trade-off) lý tưởng cho Thiết bị biên	57
4.4	Ablation Study	57
4.5	Phân tích định tính	59
4.5.1	Phân tích phân cụm đặc trưng (t-SNE)	59
4.6	Thảo luận kết quả	60
4.6.1	Phân tích điểm mạnh và hạn chế	60
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
5.1	Kết luận	61
5.2	Hạn chế của nghiên cứu	62

5.3 Hướng phát triển 63

Danh sách bảng

2.1	So sánh các kiến trúc KWS tiêu biểu trên Google Speech Commands V2 (35 lớp)	27
3.1	Thông số cấu hình trích xuất Log-Mel Spectrogram	35
3.2	Cấu trúc topo chi tiết của QANet-Lite	39
4.1	Cấu hình phần cứng sử dụng trong thực nghiệm	53
4.2	Siêu tham số huấn luyện	54
4.3	Độ phức tạp kiến trúc và hiệu năng cơ sở trên máy trạm hiệu năng cao (RTX 3070)	54
4.4	So sánh hiệu năng trên thiết bị NVIDIA Jetson Nano	55
4.5	Kết quả Ablation Study – đóng góp tích lũy của từng thành phần	58

Danh sách hình vẽ

2.1	Cấu trúc cơ bản của một Mạng nơ-ron nhân tạo đa tầng (Multilayer Perceptron), bao gồm lớp đầu vào (Input Layer), các lớp ẩn (Hidden Layers) và lớp đầu ra (Output Layer).	7
2.2	Minh họa quá trình Lan truyền ngược (Backpropagation): tính sai số tại đầu ra, truyền ngược qua các lớp và cập nhật trọng số theo gradient. . .	9
2.3	Kiến trúc tổng quan của CNN, bao gồm các bước trích xuất đặc trưng (Convolution, ReLU, Pooling) và phân loại (Flatten, Fully Connected, Softmax).	12
2.4	Minh họa các hàm kích hoạt phổ biến.	14
2.5	Quy trình STFT: phân đoạn tín hiệu bằng cửa sổ trượt và áp dụng FFT để chuyển sang miền tần số.	15
2.6	Cấu trúc Tích chập tách biệt chiều sâu (DSC): Depthwise Convolution lọc không gian trên từng kênh riêng biệt, theo sau là Pointwise Convolution 1×1 tổng hợp thông tin kênh.	19
2.7	Kiến trúc mạng MobileNetV2 với các khối Inverted Residual.	20
2.8	Cấu trúc khối Chú ý kênh (Channel Attention), sử dụng Global Average Pooling và Sigmoid để học trọng số quan trọng của từng kênh đặc trưng.	22
2.9	Cấu trúc Mô-đun Chú ý Không gian (Spatial Attention), sử dụng Average/Max Pooling theo chiều kênh kết hợp tích chập 7×7 để tập trung vào các vùng không gian quan trọng.	22
2.10	Cấu trúc khối Squeeze-and-Excitation (SE Block): Squeeze nén thông tin không gian qua GAP, Excitation học trọng số kênh qua MLP hai lớp.	23

2.11	Cấu trúc CBAM: kết hợp tuần tự Channel Attention Module và Spatial Attention Module để tinh chỉnh feature map theo cả hai chiều.	24
2.12	Cấu trúc mô hình KWS: trích xuất đặc trưng phổ tần từ tín hiệu âm thanh và đưa qua các lớp phân loại để đưa ra dự đoán từ khóa.	25
2.13	Cơ chế Chứng cất tri thức: Student được huấn luyện dựa trên cả nhãn thực tế (hard labels) và phân phối xác suất từ Teacher (soft labels). . . .	28
3.1	Kiến trúc tổng thể của hệ thống.	33
3.2	Kiến trúc mô hình Teacher (AST).	36
3.3	Kiến trúc tổng quan của QANet-Lite gồm ba giai đoạn, mỗi giai đoạn bao gồm khối QuadAxialBlock (A) và CrossAxisDownsample (B). Chi tiết QuadAxialBlock (dưới trái) với bốn nhánh song song và bộ định tuyến động; chi tiết CrossAxisDownsample (dưới phải) với cơ chế cổng chéo theo trục thời gian–tần số.	38
4.1	Biểu đồ đánh đổi giữa độ chính xác (Accuracy) và năng lượng tiêu thụ (Power) trên Jetson Nano. Kích thước bong bóng tỷ lệ thuận với Hệ số thời gian thực (RTF).	57
4.2	Biểu đồ t-SNE trực quan hóa không gian đặc trưng của mô hình QANet-Lite. Các cụm từ khóa (keywords) được phân tách rõ ràng, thể hiện khả năng trích xuất đặc trưng mang tính phân loại cao của mô hình.	59

Danh mục các từ viết tắt

Dưới đây là danh mục các từ viết tắt được sử dụng trong luận văn.

Các từ viết tắt	Định nghĩa
ANN	Artificial Neural Network (Mạng nơ-ron nhân tạo)
ASR	Automatic Speech Recognition (Nhận dạng giọng nói tổng quát)
AST	Audio Spectrogram Transformer
AUC	Area Under the ROC Curve (Diện tích dưới đường cong ROC)
BN	Batch Normalization (Chuẩn hóa theo lô)
CAM	Channel Attention Module (Mô-đun chú ý kênh)
CBAM	Convolutional Block Attention Module
ch	channels
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
DCT	Discrete Cosine Transform
DFT	Discrete Fourier Transform
DL	Deep Learning
DNN	Deep Neural Network (Mạng nơ-ron sâu)
DSC	Depthwise Separable Convolution (Tích chập tách biệt chiều sâu)
DSP	Digital Signal Processing (Xử lý tín hiệu số)
DWConv	Depthwise Convolution (Tích chập chiều sâu)

Các từ viết tắt	Định nghĩa
FFT	Fast Fourier Transform (Biến đổi Fourier nhanh)
FLOPs	Floating Point Operations Per Second (Số phép toán dấu phẩy động mỗi giây)
GAP	Global Average Pooling (Gộp trung bình toàn cục)
GMP	Global Max Pooling (Gộp cực đại toàn cục)
IoT	Internet of Things (Internet vạn vật)
IPS	Inferences Per Second (Thông lượng/Số lần suy luận mỗi giây)
KD	Knowledge Distillation (Chưng cất tri thức)
KWS	Keyword Spotting (Nhận dạng từ khóa)
MACs	Multiply-Accumulate Operations (Số phép toán nhân-cộng)
MFCC	Mel-Frequency Cepstral Coefficients
ONNX	Open Neural Network Exchange
PTQ	Post-Training Quantization (Lượng tử hóa sau huấn luyện)
PWConv	Pointwise Convolution (Tích chập điểm)
QAT	Quantization-Aware Training (Huấn luyện nhận thức lượng tử hóa)
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
RTF	Real-Time Factor (Hệ số thời gian thực)
SE	Squeeze-and-Excitation
SGD	Stochastic Gradient Descent (Hạ gradient ngẫu nhiên)

Các từ viết tắt	Định nghĩa
SNR	Signal-to-Noise Ratio (Tỷ lệ tín-nhiều)
SoC	System-on-Chip (Vi mạch hệ thống)
SSM	State Space Models (Mô hình không gian trạng thái)
STFT	Short-Time Fourier Transform (Biến đổi Fourier thời gian ngắn)
ViT	Vision Transformer

Chương 1

MỞ ĐẦU

1.1 Giới thiệu

Trong kỷ nguyên số hoá hiện nay, giao diện tương tác bằng giọng nói đang dần trở thành phương thức giao tiếp tiêu chuẩn giữa con người và máy móc. Các hệ thống điều khiển bằng giọng nói đóng vai trò nền tảng trong hệ sinh thái Internet vạn vật (Internet of Things – IoT), được ứng dụng rộng rãi từ thiết bị nhà thông minh, robot tự hành cho đến các thiết bị điện tử đeo tay [1, 2]. Bước đầu tiên và quan trọng nhất trong mọi hệ thống tương tác giọng nói là nhận dạng từ khóa (Keyword Spotting) – thành phần luôn bật (always-on) chịu trách nhiệm liên tục lắng nghe luồng âm thanh môi trường và phát hiện các lệnh kích hoạt cụ thể từ người dùng, từ đó đánh thức các hệ thống xử lý phức tạp hơn ở tầng trên [3].

Theo phương thức truyền thống, dữ liệu âm thanh được thu thập tại thiết bị đầu cuối rồi gửi lên máy chủ đám mây (cloud) để xử lý. Tuy nhiên, kiến trúc này bộc lộ nhiều hạn chế nghiêm trọng trong bối cảnh IoT quy mô lớn: độ trễ đường truyền làm giảm trải nghiệm người dùng, rủi ro rò rỉ dữ liệu cá nhân gây lo ngại về quyền riêng tư, và sự phụ thuộc hoàn toàn vào kết nối mạng liên tục không đảm bảo tính sẵn sàng của hệ thống [4,5]. Xu hướng tất yếu hiện nay là dịch chuyển quá trình tính toán xuống trực tiếp các thiết bị biên (Edge Computing). Đây vốn là nơi dữ liệu được sinh ra nhằm mục đích đảm bảo tính bảo mật, giảm thiểu độ trễ và giải phóng băng thông.

Tuy nhiên, việc đưa các mô hình học sâu xuống thiết bị biên đặt ra những thách thức

kỹ thuật hết sức khắt khe. Các thiết bị IoT thường chỉ được trang bị vài kilobyte đến vài megabyte bộ nhớ SRAM, xung nhịp xử lý hạn chế và ngân sách năng lượng cực kỳ chật hẹp [5, 6]. Thực tế cho thấy các giải pháp hiện tại thường rơi vào một trong hai thái cực: hoặc là mô hình quá lớn khiến thiết bị bị quá tải và nhanh chóng cạn kiệt năng lượng; hoặc là mô hình bị cắt giảm quá mức khiến độ chính xác nhận dạng tụt giảm nghiêm trọng, đồng thời phát sinh nghịch lý tiêu thụ năng lượng (Power Paradox) – hiện tượng do mô hình tối ưu FLOPs quá mức lại tiêu thụ điện nhiều hơn do buộc vi mạch System-on-Chip (SoC) hoạt động liên tục ở trạng thái đỉnh mà không có khoảng nghỉ vì mô [7, 8].

Nhận thấy tính cấp thiết và sự phức tạp của vấn đề, đề tài này tập trung nghiên cứu và phát triển một hệ thống KWS đạt được sự cân bằng toàn diện giữa ba yếu tố: độ chính xác cao, độ trễ thấp và tiết kiệm năng lượng. Thay vì triển khai nguyên vẹn các mô hình lớn hoặc đơn thuần cắt giảm kích thước, nghiên cứu đề xuất một kiến trúc mạng nơ-ron siêu nhẹ mang cấu trúc đường ống phân nhánh giúp điều hòa nhịp tính toán phần cứng, kết hợp cơ chế chú ý (attention mechanism) nhằm tối ưu độ chính xác phân loại. Đồng thời, kỹ thuật chưng cất tri thức (Knowledge Distillation) được áp dụng để chuyển giao biểu diễn đặc trưng từ mô hình lớn sang mô hình nhỏ gọn, hướng đến triển khai hiệu quả trên các nền tảng Edge AI trong thực tế có thể sử dụng hằng ngày.

1.2 Mục tiêu đề tài

Đề tài được thực hiện nhằm đạt được các mục tiêu cốt lõi sau:

- Đề xuất và xây dựng kiến trúc mạng học sâu siêu nhẹ (QANet-Lite) dành riêng cho bài toán KWS trên thiết bị biên, đảm bảo kích thước mô hình và số lượng tham số ở mức tối thiểu trong khi vẫn duy trì độ chính xác cạnh tranh.
- Ứng dụng kỹ thuật Chưng cất tri thức (Knowledge Distillation) để chuyển giao biểu diễn đặc trưng từ mô hình Giáo viên (Teacher) lớn sang mô hình Học sinh (Student) siêu nhẹ, cải thiện độ chính xác mà không phát sinh thêm chi phí tính toán trong giai đoạn suy luận.
- Tích hợp cơ chế chú ý (Attention Mechanism) nhằm tăng cường khả năng trích xuất

đặc trưng giọng nói và ức chế nhiễu nền, đồng thời thiết kế cấu trúc mạng phân nhánh song song giúp điều hòa nhịp tính toán phần cứng, từ đó tối ưu hóa mức tiêu thụ điện năng.

- Triển khai toàn diện hệ thống lên thiết bị phần cứng thực tế (NVIDIA Jetson Nano Developer Kit) và đo đạc, phân tích các chỉ số hoạt động trong điều kiện thời gian thực: độ trễ suy luận, thông lượng (IPS), hệ số thời gian thực (RTF) và mức tiêu thụ điện năng.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài bao gồm:

- Bài toán Nhận dạng từ khóa (KWS) trong môi trường xử lý tại biên (Edge AI), với yêu cầu hoạt động liên tục, thời gian thực và tiết kiệm năng lượng.
- Các kỹ thuật thiết kế mạng nơ-ron nhẹ: tích chập tách biệt chiều sâu (Depthwise Separable Convolution), khối thắt cổ chai (Bottleneck Block), và cơ chế chú ý (Attention Mechanism) trong phân tích quang phổ âm thanh.
- Kỹ thuật Chưng cất tri thức (Knowledge Distillation) trong bối cảnh truyền tri thức liên kiến trúc từ mô hình Transformer sang mạng CNN siêu nhẹ.
- Hiệu năng thực thi và mức tiêu thụ điện năng của vi mạch SoC khi chạy các mô hình học sâu trực tiếp trên phần cứng biên.

1.3.2 Phạm vi nghiên cứu

- **Dữ liệu nghiên cứu:** Đề tài sử dụng tập dữ liệu âm thanh mã nguồn mở Google Speech Commands V2 [9] – bộ dữ liệu chuẩn quốc tế gồm 105.829 đoạn âm thanh 1 giây thuộc 35 lớp từ khóa, được thu âm ở tần số lấy mẫu 16 kHz bởi hơn 2.000 người dùng trong nhiều điều kiện môi trường khác nhau – làm quy chuẩn huấn luyện và đánh giá.

- **Phần cứng thực nghiệm:** Việc triển khai và đánh giá hiệu năng thực tế (độ trễ, thông lượng, điện năng) được giới hạn trên thiết bị NVIDIA Jetson Nano Developer Kit vận hành ở chế độ MAXN, sử dụng tiện ích `tegrastats` để thu thập số liệu phần cứng.
- **Phạm vi mô hình:** Nghiên cứu tập trung vào giai đoạn trích xuất đặc trưng và phân loại, không bao gồm giai đoạn phát hiện điểm kích hoạt (wake word detection pipeline) hay lượng tử hoá mô hình (quantization).

1.4 Phương pháp nghiên cứu

Đề tài áp dụng kết hợp hai phương pháp nghiên cứu bổ trợ lẫn nhau:

Nghiên cứu lý thuyết: Phân tích và tổng hợp các tài liệu khoa học trong và ngoài nước về xử lý tín hiệu âm thanh, kiến trúc mạng nơ-ron học sâu cho thiết bị biên, cơ chế chú ý và chứng cất tri thức. Trên cơ sở đó, xác định khoảng trống nghiên cứu và đề xuất hướng thiết kế kiến trúc phù hợp với các ràng buộc phần cứng.

Nghiên cứu thực nghiệm: Xây dựng và huấn luyện mô hình trên máy trạm trang bị GPU NVIDIA RTX 3070 sử dụng framework PyTorch. Sau khi hoàn thiện, mô hình được chuyển đổi sang định dạng ONNX và triển khai trực tiếp lên bo mạch NVIDIA Jetson Nano. Toàn bộ các chỉ số hiệu năng – độ trễ suy luận, thông lượng, hệ số thời gian thực và mức tiêu thụ điện – được đo đạc và ghi nhận trong điều kiện hoạt động thực tế để đảm bảo tính khách quan và khả năng tái lập kết quả.

1.5 Bố cục của đề tài

Nội dung khóa luận được tổ chức thành 5 chương với bố cục như sau:

- **Chương 1 – Mở đầu:** Trình bày tính cấp thiết của bài toán, mục tiêu nghiên cứu, đối tượng, phạm vi và phương pháp thực hiện đề tài.
- **Chương 2 – Cơ sở lý thuyết:** Cung cấp nền tảng kiến thức từ tổng quát đến cụ thể: học sâu, mạng CNN, xử lý tín hiệu âm thanh, kiến trúc CNN nhẹ, cơ chế chú ý, bài toán KWS trên thiết bị biên, kỹ thuật chứng cất tri thức và các thang đo đánh giá.

- **Chương 3 – Thiết kế và thi công hệ thống:** Trình bày chi tiết kiến trúc QANet-Lite được đề xuất, pipeline trích xuất đặc trưng, chiến lược chưng cất tri thức và quy trình triển khai lên phần cứng.
- **Chương 4 – Kết quả thực hiện:** Đánh giá hiệu năng mô hình trên máy trạm, phân tích kết quả triển khai phần cứng và thực hiện ablation study để định lượng đóng góp của từng thành phần trong kiến trúc.
- **Chương 5 – Kết luận và hướng phát triển:** Tổng kết các kết quả đạt được, chỉ ra các hạn chế còn tồn tại và đề xuất hướng nghiên cứu mở rộng trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Học sâu (Deep Learning)

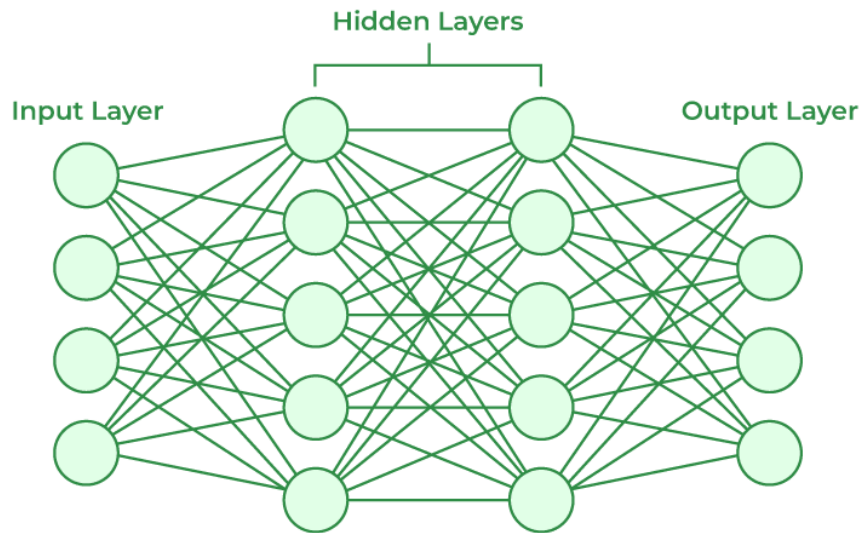
2.1.1 Mạng nơ-ron nhân tạo (Artificial Neural Networks)

Mạng nơ-ron nhân tạo (Artificial Neural Network – ANN) là một lớp mô hình học máy được lấy cảm hứng từ cơ chế xử lý thông tin của hệ thần kinh sinh học. Bộ não con người bao gồm hàng tỷ tế bào thần kinh (neuron) liên kết với nhau qua các khớp thần kinh (synapse); mỗi tế bào nhận tín hiệu từ nhiều tế bào khác, tổng hợp và phát ra tín hiệu mới nếu tổng vượt ngưỡng kích hoạt. ANN mô hình hoá cơ chế này bằng các phép toán tuyến tính và phi tuyến có thể vi phân được, từ đó cho phép tối ưu hoá bằng giải tích.

Một ANN được tổ chức thành các lớp: lớp đầu vào (input layer) nhận dữ liệu thô, một hoặc nhiều lớp ẩn (hidden layers) thực hiện biến đổi đặc trưng, và lớp đầu ra (output layer) tạo ra dự đoán cuối cùng. Mỗi nơ-ron nhận tổng có trọng số của các đầu vào từ lớp trước, cộng thêm hệ số bias, rồi áp dụng hàm kích hoạt phi tuyến:

$$a^{(l)} = f\left(\mathbf{W}^{(l)} a^{(l-1)} + \mathbf{b}^{(l)}\right) \quad (2.1)$$

trong đó $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ là ma trận trọng số của lớp l , $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ là véc-tơ bias, $a^{(l-1)}$ là đầu ra của lớp trước, và $f(\cdot)$ là hàm kích hoạt phi tuyến. Khi số lớp ẩn tăng lên (thường từ hai lớp trở lên), mạng được gọi là *mạng nơ-ron sâu* (Deep Neural Network – DNN).



Hình 2.1: Cấu trúc cơ bản của một Mạng nơ-ron nhân tạo đa tầng (Multilayer Perceptron), bao gồm lớp đầu vào (Input Layer), các lớp ẩn (Hidden Layers) và lớp đầu ra (Output Layer).

Trong thực tiễn, *chiều sâu* (depth) của mạng đóng vai trò quan trọng hơn *chiều rộng* (width): các DNN sâu với ít tham số hơn có thể biểu diễn các hàm mà mạng nông cần số lượng nơ-ron theo hàm mũ mới biểu diễn được [10]. Nhờ vào sự xếp chồng của nhiều lớp phi tuyến, DNN có khả năng học được các biểu diễn đặc trưng phân cấp (hierarchical representations) – từ các đặc trưng cấp thấp (cạnh, góc, giai điệu cơ bản trong âm thanh) đến các đặc trưng cấp cao (hình dạng vật thể, từ khóa trong tiếng nói) – trực tiếp từ dữ liệu thô mà không cần thiết kế đặc trưng thủ công.

Các hàm kích hoạt

Hàm kích hoạt quyết định tính phi tuyến của mạng và ảnh hưởng trực tiếp đến động lực huấn luyện. Nếu không có hàm kích hoạt phi tuyến, toàn bộ mạng dù nhiều lớp vẫn chỉ tương đương một phép biến đổi tuyến tính duy nhất – mất đi toàn bộ năng lực biểu diễn. Các hàm kích hoạt phổ biến trong học sâu hiện đại bao gồm:

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. ReLU trở thành hàm kích hoạt phổ biến nhất nhờ ba ưu điểm chính: tính toán cực kỳ đơn giản (chỉ là phép so sánh và cắt ngưỡng); gradient là hằng số 1 cho $x > 0$, giảm thiểu hiện tượng triệt tiêu gradient so với sigmoid/tanh; và tạo ra kích hoạt thưa (sparse activation) khi nhiều nơ-ron không kích

hoạt, giúp mô hình tổng quát hoá tốt hơn. Nhược điểm của ReLU là hiện tượng “nơ-ron chết” (dying ReLU): một khi nơ-ron rơi vào vùng âm, gradient bằng 0 và nơ-ron không bao giờ được cập nhật.

ReLU6: $f(x) = \min(\max(0, x), 6)$. Biến thể giới hạn đầu ra trong $[0, 6]$, được đề xuất trong MobileNet [11]. Giới hạn trên 6 không làm giảm đáng kể năng lực biểu diễn (giá trị kích hoạt thực tế hiếm khi vượt 6 sau chuẩn hoá), nhưng có lợi thế rõ rệt khi lượng tử hoá sang số nguyên 8-bit (int8): phạm vi $[0, 6]$ được chia đều cho 255 mức, cho độ phân giải lượng tử tốt hơn so với phạm vi không giới hạn. Đây là lý do ReLU6 trở thành lựa chọn mặc định trong các mạng thiết kế cho thiết bị nhúng.

Sigmoid: $f(x) = 1/(1 + e^{-x})$, ánh xạ đầu vào về khoảng $(0, 1)$. Tuy không còn được dùng làm hàm kích hoạt lớp ẩn (do gradient triệt tiêu ở vùng bão hoà), sigmoid vẫn giữ vai trò quan trọng trong các cơ chế cổng (gating) và cơ chế chú ý (attention) vì đầu ra trong $(0, 1)$ được diễn giải tự nhiên như trọng số xác suất.

Hard-Sigmoid và Hard-Swish: Các xấp xỉ tuyến tính từng đoạn của sigmoid và swish, được thiết kế cho phần cứng không có đơn vị tính toán hàm mũ. Hard-sigmoid: $f(x) = \text{clip}((x + 3)/6, 0, 1)$; Hard-swish: $f(x) = x \cdot \text{Hard-sigmoid}(x)$. Cả hai được sử dụng trong MobileNetV3 [12] để tăng độ chính xác với chi phí tính toán tối thiểu trên phần cứng nhúng.

2.1.2 Quá trình huấn luyện và tối ưu hoá

Lan truyền ngược (Backpropagation)

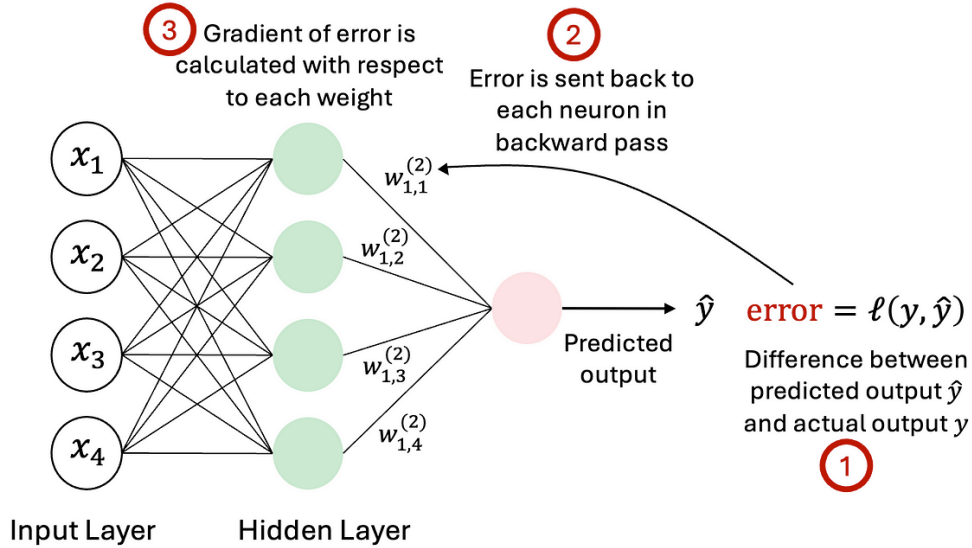
Huấn luyện ANN là quá trình tối ưu hoá tập tham số $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^L$ để cực tiểu hoá hàm mất mát $\mathcal{L}(\theta)$ đo độ lệch giữa dự đoán và nhãn thực. Đây là bài toán tối ưu hoá phi lồi trong không gian có số chiều rất lớn, không có giải pháp dạng đóng.

Thuật toán lan truyền ngược (Backpropagation) [13] giải quyết bài toán này bằng quy tắc dây chuyền (chain rule), gồm hai pha:

Lan truyền xuôi (Forward pass): Tính toán tuần tự từ lớp đầu vào đến lớp đầu ra, lưu lại các giá trị kích hoạt trung gian $a^{(l)}$ tại mỗi lớp để dùng trong bước lan truyền ngược.

Lan truyền ngược (Backward pass): Tính gradient của hàm mất mát theo từng tham

số bằng quy tắc dây chuyền, bắt đầu từ lớp đầu ra và lan truyền ngược về lớp đầu vào. Gradient tại lớp l phụ thuộc vào gradient từ lớp $l + 1$ và đạo hàm của hàm kích hoạt tại lớp đó, tạo nên chuỗi nhân liên tiếp xuyên suốt toàn mạng.



Hình 2.2: Minh họa quá trình Lan truyền ngược (Backpropagation): tính sai số tại đầu ra, truyền ngược qua các lớp và cập nhật trọng số theo gradient.

Sau khi tính gradient, các tham số được cập nhật theo quy tắc hạ gradient ngẫu nhiên (Stochastic Gradient Descent – SGD):

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L} \quad (2.2)$$

trong đó $\eta > 0$ là tốc độ học (learning rate). Trong thực tế, gradient được tính trên từng mini-batch gồm B mẫu thay vì toàn bộ tập huấn luyện, cân bằng giữa độ chính xác ước lượng gradient và tốc độ cập nhật.

Một vấn đề quan trọng trong mạng sâu là *triệt tiêu gradient* (vanishing gradient): khi gradient lan truyền qua nhiều lớp, tích của nhiều đạo hàm nhỏ hơn 1 khiến gradient về gần 0, làm các lớp gần đầu vào học rất chậm. Đây là nguyên nhân chính khiến sigmoid/tanh bị thay thế bởi ReLU, và là lý do các kỹ thuật như kết nối tắt (skip connection) và batch normalization trở nên thiết yếu trong mạng sâu hiện đại.

Các thuật toán tối ưu hoá nâng cao

SGD với động lượng (Momentum): Tích lũy gradient theo hướng nhất quán qua các bước cập nhật, giúp vượt qua các điểm yên ngựa (saddle points) và giảm dao động xung quanh hướng tối ưu:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\theta} \mathcal{L}_t, \quad \theta \leftarrow \theta - \eta \mathbf{v}_t \quad (2.3)$$

trong đó $\beta \approx 0.9$ là hệ số quán tính. Momentum đặc biệt hiệu quả khi gradient có hướng nhất quán qua nhiều bước liên tiếp – tương tự hiệu ứng quán tính vật lý giúp “lăn xuống dốc” nhanh hơn.

Adam (Adaptive Moment Estimation) [14] là thuật toán được sử dụng rộng rãi nhất trong học sâu hiện đại, kết hợp ý tưởng của momentum với điều chỉnh learning rate thích nghi cho từng tham số. Adam duy trì ước lượng moment bậc nhất $\mathbf{m}_t$ (trung bình gradient) và moment bậc hai $\mathbf{v}_t$ (phương sai gradient), sau đó hiệu chỉnh bias để tránh sai lệch trong giai đoạn đầu:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad (2.4)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (2.5)$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t \quad (2.6)$$

Các siêu tham số mặc định $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ hoạt động tốt trong hầu hết bài toán mà không cần tinh chỉnh. Ưu điểm nổi bật của Adam là hội tụ nhanh ngay cả với gradient nhiễu và trong không gian tham số không đồng nhất – đặc biệt hữu ích khi huấn luyện mạng CNN nhẹ trên tập dữ liệu âm thanh có phân phối không đều.

Các kỹ thuật điều chuẩn

Batch Normalization (BN) [15] chuẩn hoá phân phối đầu vào của mỗi lớp theo từng mini-batch, giải quyết vấn đề dịch chuyển phân phối nội bộ (internal covariate shift) – hiện tượng phân phối đầu vào mỗi lớp thay đổi liên tục trong quá trình huấn luyện. Cụ

thể, BN chuẩn hoá về trung bình 0 và phương sai 1, rồi học lại tham số scale γ và shift β :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (2.7)$$

trong đó μ_B và σ_B^2 là trung bình và phương sai của mini-batch. BN cho phép sử dụng learning rate cao hơn, giảm phụ thuộc vào khởi tạo trọng số, và có tác dụng điều chuẩn nhẹ. Trong giai đoạn suy luận, μ và σ được thay bằng các thống kê di động tích lũy trong huấn luyện.

Dropout [16] ngẫu nhiên tắt (set về 0) một tỷ lệ p nơ-ron trong mỗi bước huấn luyện, tương đương với huấn luyện đồng thời một tập hợp mạng con khác nhau và lấy trung bình dự đoán khi suy luận. Dropout buộc mạng học các đặc trưng độc lập, tránh đồng thích nghi (co-adaptation) giữa các nơ-ron, giảm overfitting đặc biệt hiệu quả khi mô hình nhỏ được huấn luyện trên tập dữ liệu hữu hạn.

Weight Decay và LR Scheduler: Thêm số hạng phạt $\lambda \|\theta\|_2^2$ vào hàm mất mát giới hạn độ phức tạp mô hình. Trong Adam, weight decay được triển khai theo dạng decoupled (AdamW) [17] để tránh tương tác bất lợi với adaptive learning rate. Giảm dần learning rate theo thời gian (Cosine Annealing, ReduceLROnPlateau) giúp hội tụ tới vùng cực tiểu ổn định hơn – chiến lược Warmup + Cosine bắt đầu từ learning rate nhỏ rồi tăng dần trước khi giảm theo cosine đặc biệt hữu ích với Adam.

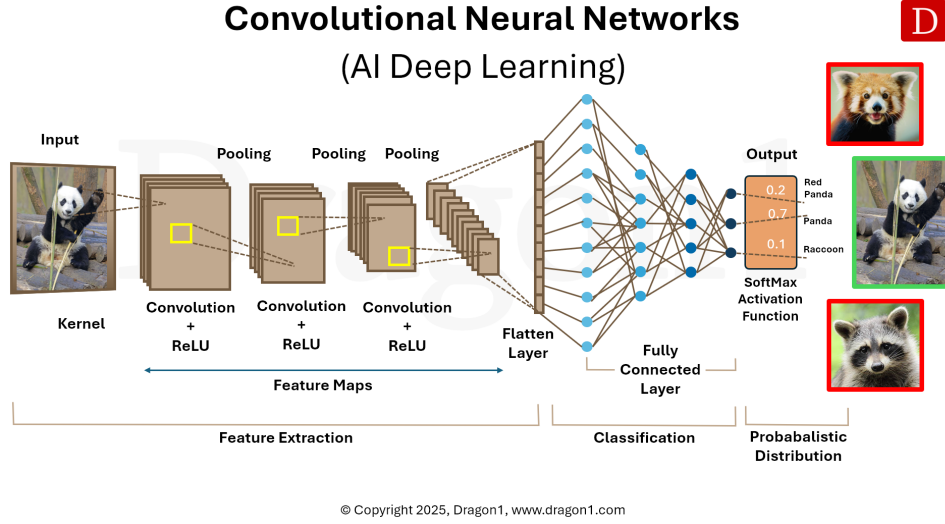
2.2 Mạng nơ-ron tích chập (Convolutional Neural Networks)

2.2.1 Nguyên lý tích chập và chia sẻ trọng số

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) là kiến trúc chuyên biệt được thiết kế để khai thác cấu trúc lưới (grid structure) của dữ liệu như ảnh hai chiều và quang phổ âm thanh. Trong khi mạng fully connected coi mỗi điểm ảnh là một chiều độc lập – bỏ qua hoàn toàn mối quan hệ không gian – CNN khai thác hai giả định cấu trúc quan trọng:

Tính cục bộ (locality): Các đặc trưng thị giác (cạnh, góc) và âm học (âm vị, formant) chỉ phụ thuộc vào vùng lân cận không gian hẹp, không phải toàn bộ đầu vào.

Bất biến dịch chuyển (translation invariance): Cùng một đặc trưng (ví dụ: âm vị /k/) xuất hiện ở nhiều vị trí thời gian-tần số khác nhau trong spectrogram nhưng cần được nhận diện như nhau, bất kể vị trí.



Hình 2.3: Kiến trúc tổng quan của CNN, bao gồm các bước trích xuất đặc trưng (Convolution, ReLU, Pooling) và phân loại (Flatten, Fully Connected, Softmax).

CNN thực thi hai giả định này thông qua *chia sẻ trọng số*: cùng một bộ lọc $\mathbf{W} \in \mathbb{R}^{k \times k \times C_{in}}$ được áp dụng tại mỗi vị trí trong không gian đầu vào. So với lớp fully connected cần $d_{in} \times d_{out}$ tham số, lớp tích chập chỉ cần $k^2 \times C_{in} \times C_{out}$ tham số – giảm hàng nghìn lần. Phép tích chập 2D được định nghĩa:

$$\mathbf{O}_c(i, j) = \sum_{c'=1}^{C_{in}} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{W}_{c,c'}(p, q) \cdot \mathbf{F}_{c'}(i+p, j+q) + b_c \quad (2.8)$$

trong đó $\mathbf{F}_{c'}$ là kênh c' của feature map đầu vào, $\mathbf{O}_c$ là kênh c của feature map đầu ra. Số phép nhân-cộng (MACs) cho một lớp tích chập:

$$\text{MACs} = C_{in} \cdot C_{out} \cdot k^2 \cdot H' \cdot W' \quad (2.9)$$

là cơ sở để so sánh và tối ưu chi phí tính toán giữa các kiến trúc.

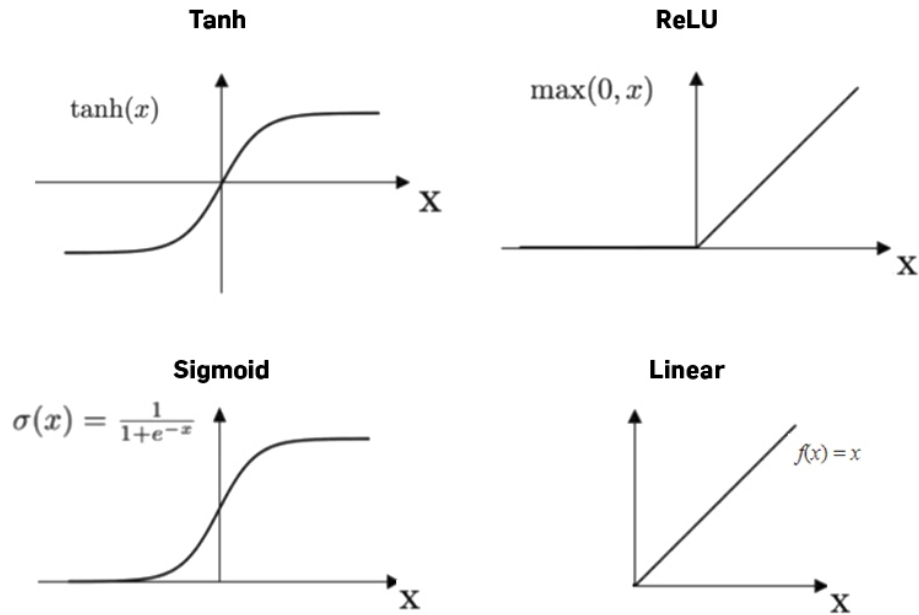
2.2.2 Các thành phần cơ bản của CNN

Một khối CNN hiện đại thường gồm chuỗi Conv–BN–ReLU, trong đó lớp tích chập thực hiện trích xuất đặc trưng, BN ổn định phân phối kích hoạt, và ReLU tạo phi tuyến. Ngoài các thành phần này, một số cấu trúc quan trọng khác bao gồm:

Lớp gộp (Pooling): Giảm kích thước không gian của feature map, tăng vùng thụ cảm cho các lớp sau và giảm chi phí tính toán. Max Pooling giữ đặc trưng mạnh nhất trong vùng, nhạy cảm với sự hiện diện của đặc trưng; Average Pooling tính trung bình năng lượng, mượt mà và ổn định hơn. *Global Average Pooling* (GAP) thu gọn toàn bộ feature map $H \times W$ về một giá trị vô hướng trên mỗi kênh – Lin et al. [18] đề xuất GAP như cách thay thế lớp FC tại đầu ra mạng, giảm đáng kể số tham số, chống overfitting và tăng khả năng diễn giải.

Kết nối tắt (Skip Connection): Đề xuất bởi He et al. trong ResNet [19], cộng trực tiếp đầu vào vào đầu ra: $\mathbf{F}_{out} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. Thay vì học toàn bộ ánh xạ mong muốn $\mathcal{H}(\mathbf{x})$, mạng chỉ cần học phần dư $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$, dễ tối ưu hơn. Kết nối tắt còn tạo đường dẫn gradient trực tiếp qua các lớp, giải quyết vấn đề triệt tiêu gradient trong mạng rất sâu.

Kiến trúc đa nhánh: Trong phân tích quang phổ âm thanh, các mẫu âm học xuất hiện ở nhiều tỉ lệ không gian-thời gian khác nhau: chi tiết âm vị cục bộ đòi hỏi bộ lọc nhỏ (3×3), trong khi cấu trúc hài hoà rộng hơn cần bộ lọc lớn hơn (5×5 hoặc 1×11). Kiến trúc đa nhánh xử lý đầu vào song song với nhiều kích thước kernel rồi kết hợp kết quả, cho phép nắm bắt đặc trưng đa tỉ lệ trong cùng một lớp theo tinh thần của Inception [20]. Hai chiến lược kết hợp phổ biến là nối ghép kênh (concatenation) – bảo toàn toàn bộ thông tin nhưng tăng chiều kênh – và cộng phần tử (element-wise addition) – dung hợp ngữ nghĩa mà không tăng chi phí.



Hình 2.4: Minh họa các hàm kích hoạt phổ biến.

2.3 Xử lý tín hiệu âm thanh

2.3.1 Biểu diễn tín hiệu âm thanh số

Âm thanh trong thực tế là sóng áp suất không khí liên tục theo thời gian. Trong hệ thống số, tín hiệu được lấy mẫu tại tần số f_s (Hz) tạo thành dãy rời rạc $x[n] = x(n/f_s)$. Theo định lý Nyquist-Shannon, tần số lấy mẫu phải ít nhất gấp đôi tần số cao nhất để tránh aliasing: $f_s \geq 2f_{max}$. Giọng nói con người chứa thành phần tần số lên đến khoảng 8 kHz, do đó tần số lấy mẫu 16 kHz là lựa chọn phổ biến trong nhận dạng giọng nói – đủ bao phủ toàn bộ dải thoại theo định lý Nyquist.

Tín hiệu âm thanh thô không phù hợp để đưa trực tiếp vào CNN 2D vì hai lý do: CNN 2D được thiết kế cho dữ liệu có cấu trúc không gian hai chiều; và trong miền thời gian, thông tin về tần số – đặc trưng thiết yếu cho nhận dạng tiếng nói – bị ẩn hoàn toàn. Do đó, bước tiền xử lý biến đổi tín hiệu từ miền thời gian sang biểu diễn thời gian-tần số là không thể thiếu.

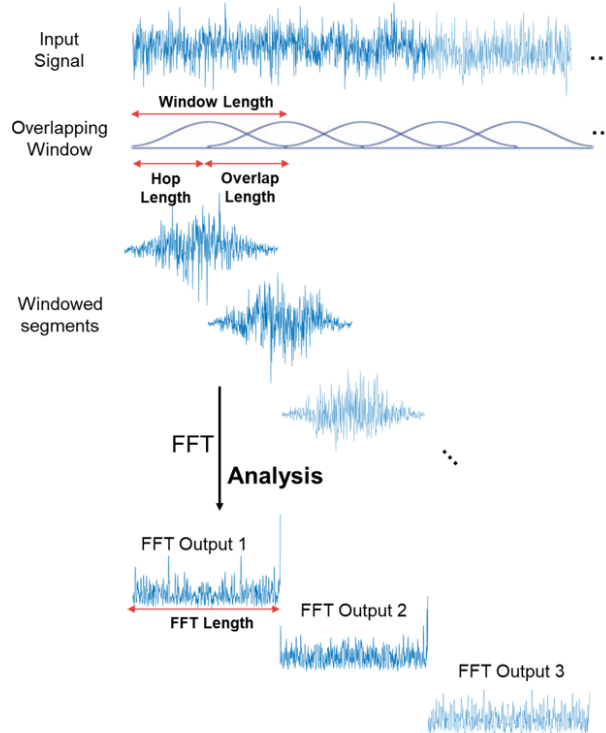
2.3.2 Biến đổi Fourier thời gian ngắn (STFT)

Biến đổi Fourier thời gian ngắn (Short-Time Fourier Transform – STFT) phân tích đồng thời thông tin thời gian và tần số dựa trên giả định tính *dừng cục bộ*: mặc dù giọng nói không dừng trên toàn đoạn phát âm, nhưng trong khoảng thời gian rất ngắn (10–30 ms), đặc tính phổ không thay đổi đáng kể.

STFT chia tín hiệu thành các khung ngắn có độ dài N mẫu, trượt với bước H mẫu (hop length), rồi áp dụng DFT cho từng khung sau khi nhân với hàm cửa sổ $w[n]$:

$$X[m, k] = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (2.10)$$

trong đó m là chỉ số khung thời gian, k là chỉ số bin tần số, và $X[m, k] \in \mathbb{C}$ là hệ số Fourier phức. Tần số tương ứng với bin k là $f_k = k \cdot f_s/N$.



Hình 2.5: Quy trình STFT: phân đoạn tín hiệu bằng cửa sổ trượt và áp dụng FFT để chuyển sang miền tần số.

Hàm cửa sổ: Mục đích của hàm cửa sổ $w[n]$ là giảm rò rỉ phổ (spectral leakage) – artefact xảy ra khi cắt khung không phải tại đúng chu kỳ tín hiệu. Cửa sổ Hann $w[n] = 0.5(1 - \cos(2\pi n/N))$ là lựa chọn phổ biến nhất: giảm rò rỉ tốt trong khi chỉ mất một

phần nhỏ năng lượng tại hai biên. Phổ công suất thu được bằng cách loại bỏ pha và giữ lại năng lượng: $P[m, k] = |X[m, k]|^2$.

Trong bài toán KWS với âm thanh 1 giây lấy mẫu 16 kHz, các tham số điển hình là: độ dài khung $N = 400$ mẫu (25 ms) – đủ dài để có phân giải tần số tốt nhưng đủ ngắn để giả định dừng cục bộ hợp lý; bước trượt $H = 160$ mẫu (10 ms) – tạo ra 99 khung thời gian; và kích thước FFT $N_{FFT} = 512$ với zero-padding để tăng độ phân giải tần số.

2.3.3 Log-Mel Spectrogram

Thang đo Mel và Mel Filterbank

Hệ thống thính giác con người cảm nhận tần số theo thang phi tuyến: nhạy cảm với sự khác biệt tần số ở vùng thấp và kém nhạy dần ở vùng cao. Thang Mel mô hình hoá đặc tính này:

$$f_{\text{mel}} = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (2.11)$$

Thang Mel xấp xỉ tốt sự phân bố dọc theo ốc tai trong tai người, nơi tần số thấp chiếm nhiều diện tích hơn tần số cao.

Một bộ gồm M bộ lọc băng thông tam giác phân bố đều trên thang Mel (không đều trên thang Hz) tạo thành Mel filterbank. Năng lượng của khung m trong bộ lọc thứ i :

$$S[m, i] = \sum_{k=0}^{N/2} P[m, k] \cdot H_i[k], \quad i = 1, \dots, M \quad (2.12)$$

Các bộ lọc Mel ở tần số thấp hẹp hơn (tính theo Hz) và ở tần số cao rộng hơn, phản ánh đúng độ nhạy thính giác. Kết quả là ma trận $\mathbf{S} \in \mathbb{R}^{T \times M}$.

Logarithm hoá và chuẩn hoá

Năng lượng âm thanh thực tế có dải động rất rộng. Áp dụng logarithm nén dải động này về phạm vi xử lý được, đồng thời phù hợp với cách con người cảm nhận độ to theo định luật Weber-Fechner:

$$\text{LogMel}[m, i] = \log(S[m, i] + \epsilon) \quad (2.13)$$

trong đó $\varepsilon > 0$ (thường $= 10^{-6}$) tránh logarithm của 0.

Kết quả là tensor 2D Log-Mel Spectrogram $\mathbf{X} \in \mathbb{R}^{T \times M}$ – thường kích thước 128×128 với âm thanh 1 giây lấy mẫu 16 kHz – có thể được CNN xử lý như ảnh đơn kênh. Toàn bộ tập dữ liệu được chuẩn hoá về trung bình 0 và độ lệch chuẩn 1 để ổn định huấn luyện và tránh một số kênh Mel chiếm ưu thế.

Log-Mel được ưu tiên hơn MFCC (Mel-Frequency Cepstral Coefficients) trong các hệ thống dựa trên CNN vì: biến đổi DCT trong MFCC phân tán thông tin cục bộ ra toàn bộ hệ số, trong khi log-Mel giữ nguyên cấu trúc không gian 2D mà CNN có thể khai thác hiệu quả bằng tích chập cục bộ.

2.3.4 Tăng cường dữ liệu âm thanh

Do điều kiện thu âm thực tế đa dạng, các kỹ thuật tăng cường dữ liệu (data augmentation) mô phỏng sự đa dạng này trong quá trình huấn luyện để cải thiện khả năng tổng quát hoá của mô hình KWS.

Tăng cường miền thời gian

Thêm nhiễu nền: Pha trộn tín hiệu sạch với đoạn nhiễu nền thực tế (tiếng ồn đường phố, quán cà phê, v.v.) theo tỷ lệ SNR ngẫu nhiên trong $[0, 15]$ dB. Phương pháp này trực tiếp mô phỏng điều kiện môi trường thực tế và là kỹ thuật hiệu quả nhất cho KWS.

Dịch chuyển thời gian: Dịch tín hiệu theo trục thời gian trong khoảng ± 100 ms. Giúp mô hình bất biến với việc từ khóa xuất hiện ở vị trí khác nhau trong khung 1 giây.

Thay đổi tốc độ và cao độ: Thay đổi tốc độ phát âm trong $[0.9, 1.1]$ và cao độ trong ± 2 semitone, mô phỏng sự đa dạng giọng nói giữa các người nói.

Tăng cường miền spectrogram

SpecAugment [21]: Che ngẫu nhiên các dải liên tiếp trên trục tần số (frequency masking) và trục thời gian (time masking) của spectrogram. SpecAugment buộc mô hình học phân loại dựa trên thông tin phân tán, không phụ thuộc vào dải tần số hay khoảng khắc cụ thể nào – tương tự Dropout nhưng có cấu trúc không gian. Đây là một

trong những kỹ thuật augmentation phổ biến nhất trong nhận dạng tiếng nói hiện đại.

Mixup [22]: Tạo mẫu tổng hợp bằng nội suy tuyến tính giữa hai cặp mẫu-nhãn ngẫu nhiên:

$$\tilde{\mathbf{X}} = \lambda \mathbf{X}_i + (1 - \lambda) \mathbf{X}_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j \quad (2.14)$$

với $\lambda \sim \text{Beta}(\alpha, \alpha)$, thường $\alpha = 0.2$ đến 1.0 . Mixup tạo ra nhãn mềm buộc mô hình học ranh giới quyết định mượt mà thay vì ranh giới sắc nét dễ overfit, đặc biệt hiệu quả với kiến trúc nhỏ gọn trên tập dữ liệu cỡ trung bình như Google Speech Commands.

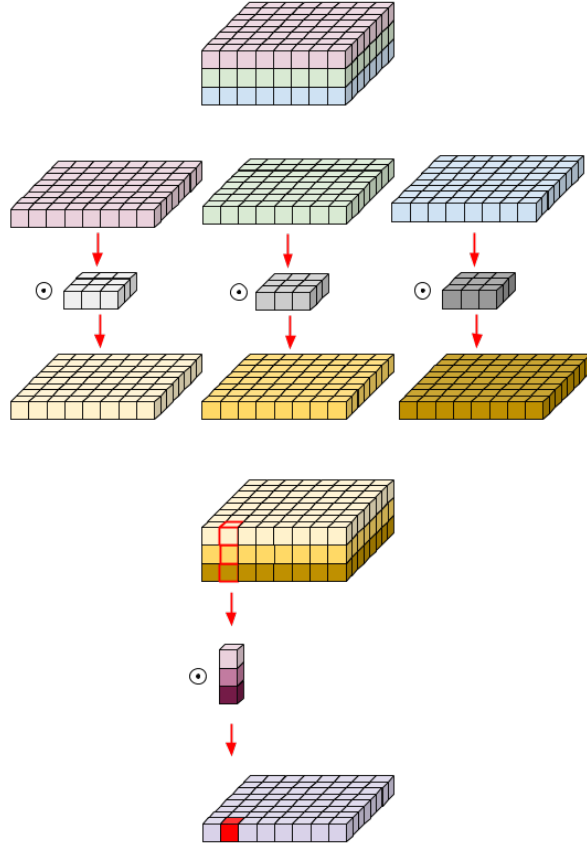
2.4 Kiến trúc CNN nhẹ cho thiết bị nhúng

2.4.1 Động lực thiết kế mạng nhẹ

Các mạng CNN đạt hiệu năng cao như ResNet-50, EfficientNet hay Audio Spectrogram Transformer đều có hàng chục triệu tham số và yêu cầu hàng trăm MB bộ nhớ – hoàn toàn vượt ngoài khả năng của thiết bị nhúng. Bài toán thiết kế mạng nhẹ đặt ra yêu cầu tìm kiếm kiến trúc đạt độ chính xác chấp nhận được trong khi duy trì ràng buộc nghiêm ngặt về số tham số ($< 1\text{M}$), FLOPs ($< 100\text{M}$), và bộ nhớ ($< 5\text{ MB}$). Ba hướng tiếp cận chính: thiết kế thủ công các toán tử hiệu quả; tìm kiếm kiến trúc tự động (NAS); và nén mô hình qua lượng tử hoá, cắt tỉa, và chưng cất tri thức.

2.4.2 Tích chập tách biệt chiều sâu (Depthwise Separable Convolution)

Tích chập tách biệt chiều sâu (Depthwise Separable Convolution – DSC) [11] là kỹ thuật nền tảng của các mạng CNN nhẹ. Ý tưởng cốt lõi là phân rã tích chập tiêu chuẩn – vừa lọc không gian vừa kết hợp kênh cùng lúc – thành hai bước tách biệt, mỗi bước chỉ thực hiện một nhiệm vụ.



Hình 2.6: Cấu trúc Tích chập tách biệt chiều sâu (DSC): Depthwise Convolution lọc không gian trên từng kênh riêng biệt, theo sau là Pointwise Convolution 1×1 tổ hợp thông tin kênh.

Depthwise Convolution (DWConv): Áp dụng một bộ lọc $k \times k$ riêng biệt cho từng kênh đầu vào, thực hiện lọc không gian mà không trao đổi thông tin giữa các kênh. Số kênh đầu ra bằng đầu vào ($C_{out} = C_{in}$), chi phí tính toán: $MACs_{DW} = C_{in} \cdot k^2 \cdot H' \cdot W'$.

Pointwise Convolution (PWConv): Áp dụng bộ lọc 1×1 để tổ hợp thông tin các kênh tại từng vị trí không gian – không có lọc không gian. Chi phí tính toán: $MACs_{PW} = C_{in} \cdot C_{out} \cdot H' \cdot W'$.

Tỷ lệ giảm chi phí tính toán so với tích chập tiêu chuẩn:

$$\frac{MACs_{DSC}}{MACs_{Std}} = \frac{1}{C_{out}} + \frac{1}{k^2} \quad (2.15)$$

Với $k = 3$ và $C_{out} = 64$: tỷ lệ giảm $\approx 1/9$, tức giảm xấp xỉ 8–9 lần cả chi phí tính toán lẫn số tham số. Đây là lý do DSC trở thành toán tử nền tảng trong toàn bộ họ mạng

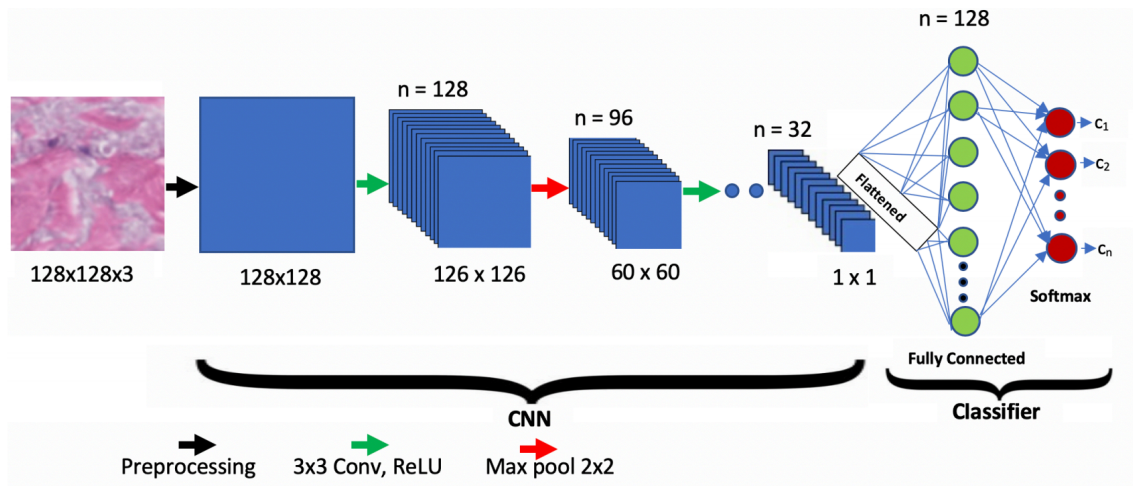
MobileNet và hầu hết các kiến trúc KWS nhẹ hiện đại.

2.4.3 Khối Inverted Residual (MobileNetV2)

MobileNetV2 [23] đề xuất khối *Inverted Residual* – đảo ngược bottleneck thông thường bằng cách dùng không gian ẩn rộng ở giữa thay vì hẹp – gồm ba bước liên tiếp:

1. *Expansion*: Tích chập 1×1 mở rộng từ C_{in} lên $t \cdot C_{in}$ kênh (hệ số mở rộng $t = 6$).
2. *Depthwise*: Tích chập DW 3×3 lọc không gian ở không gian rộng $t \cdot C_{in}$ kênh.
3. *Projection*: Tích chập 1×1 chiếu về C_{out} kênh – không có hàm kích hoạt sau bước này để tránh mất thông tin ở không gian chiều thấp.

Kết nối tắt được thêm khi $C_{in} = C_{out}$ (stride = 1). Thiết kế này giữ biểu diễn chiều thấp ở đầu vào/đầu ra (tiết kiệm bộ nhớ) trong khi vẫn cho DWConv học biểu diễn không gian phong phú ở không gian mở rộng trung gian. MobileNetV3 [12] tiếp tục cải tiến bằng cách tích hợp Squeeze-and-Excitation và hard activation để đạt hiệu năng tốt hơn cùng chi phí tính toán.



Hình 2.7: Kiến trúc mạng MobileNetV2 với các khối Inverted Residual.

2.4.4 Nghịch lý hiệu quả năng lượng (Power Paradox)

Một quan niệm phổ biến là giảm FLOPs lý thuyết đồng nghĩa với tiêu thụ điện năng thấp hơn trên phần cứng thực tế. Tuy nhiên, các nghiên cứu thực nghiệm [7, 8] chứng

minh điều ngược lại có thể xảy ra – được gọi là *Power Paradox*.

Khi mô hình được tối ưu FLOPs quá mức (ví dụ: sử dụng quá nhiều DWConv với tỷ lệ tính toán/bộ nhớ thấp), mỗi lần suy luận hoàn thành rất nhanh, buộc SoC hoạt động liên tục ở trạng thái đỉnh mà không có khoảng nghỉ vi mô (micro-idling). Năng lượng mỗi lần suy luận:

$$E_{inf} = \frac{P_{avg}}{\text{Throughput (IPS)}} \quad (2.16)$$

Nghịch lý xảy ra khi mô hình A có FLOPs thấp hơn B, nhưng throughput cao hơn dẫn đến P_{avg} tăng theo tỷ lệ, khiến E_{inf} của A thực tế cao hơn B.

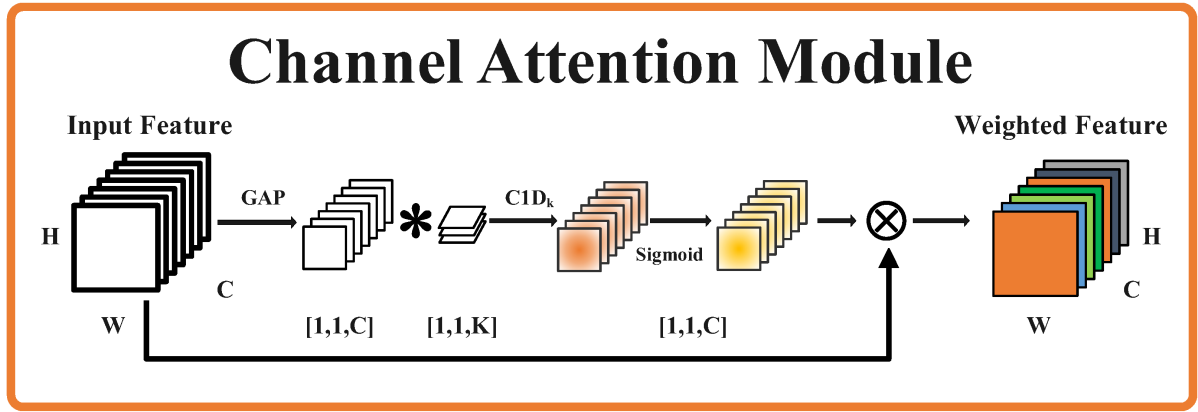
2.5 Cơ chế chú ý (Attention Mechanism)

2.5.1 Động lực và phân loại

Trong mạng CNN thông thường, tất cả vị trí không gian và kênh đặc trưng đóng góp như nhau vào bước tính toán tiếp theo – mạng không có khả năng tập trung vào vùng quan trọng. Trong quang phổ âm thanh, điều này là bất lợi rõ ràng: phần nhiều nền chiếm phần lớn năng lượng nhưng không mang thông tin về từ khóa; dải tần số giọng nói (300–3400 Hz) và khoảng thời gian chứa âm vị chủ chốt mới thực sự quan trọng cho bài toán nhận dạng.

Cơ chế chú ý giải quyết vấn đề này bằng cách học một hàm tái cân bằng – dựa trên nội dung của feature map – để khuếch đại các vùng quan trọng và ức chế các vùng nhiễu. Về mặt toán học, cơ chế chú ý học một bản đồ trọng số $\mathbf{M} \geq 0$ rồi nhân điểm với feature map: $\mathbf{F}' = \mathbf{M} \otimes \mathbf{F}$. Theo chiều tái cân bằng, attention trong CNN phân thành:

- **Chú ý kênh (Channel Attention):** Học vector trọng số theo chiều kênh, trả lời câu hỏi “*đặc trưng/kênh nào quan trọng?*”.
- **Chú ý không gian (Spatial Attention):** Học bản đồ trọng số theo chiều không gian $H \times W$, trả lời câu hỏi “*vị trí thời gian-tần số nào quan trọng?*”.
- **Chú ý kết hợp:** Kết hợp cả hai, thường theo dạng tuần tự hoặc song song.



Hình 2.8: Cấu trúc khối Chú ý kênh (Channel Attention), sử dụng Global Average Pooling và Sigmoid để học trọng số quan trọng của từng kênh đặc trưng.

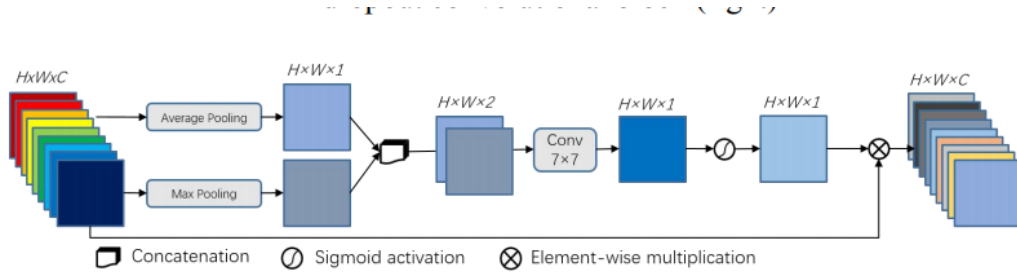


Fig. 4. Diagram of the Spatial Attention Module

Hình 2.9: Cấu trúc Mô-đun Chú ý Không gian (Spatial Attention), sử dụng Average/Max Pooling theo chiều kênh kết hợp tích chập 7×7 để tập trung vào các vùng không gian quan trọng.

2.5.2 Squeeze-and-Excitation (SE)

SENet [24] là kiến trúc tiên phong cho chú ý kênh, giành vị trí đầu trong ILSVRC 2017. Ý tưởng là dùng toàn bộ ngữ cảnh không gian (toàn bộ feature map) để học trọng số chú ý kênh, thông qua hai bước tường minh:

Bước Squeeze: Tóm tắt đặc trưng không gian của mỗi kênh thành một số vô hướng bằng Global Average Pooling:

$$z_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W F_c(i, j), \quad c = 1, \dots, C \quad (2.17)$$

Véc-tơ $\mathbf{z} \in \mathbb{R}^C$ mã hoá thông tin “toàn cảnh” của từng kênh.

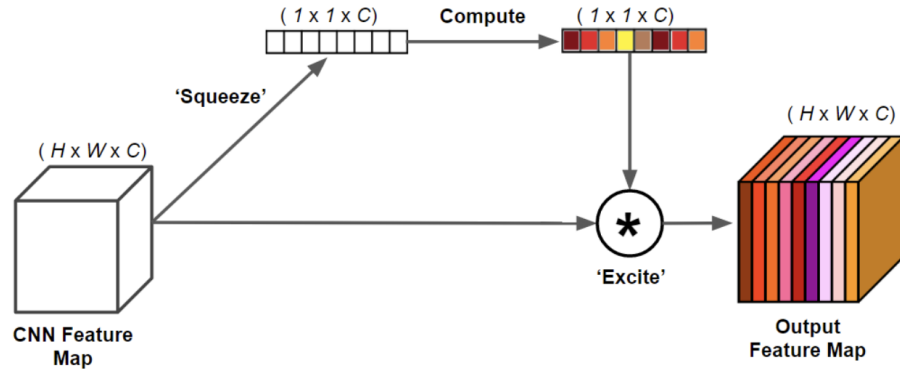
Bước Excitation: Học hàm phi tuyến ánh xạ từ biểu diễn nén sang trọng số kênh qua MLP hai lớp với hệ số nén r (thường $r = 16$):

$$\mathbf{m}_c = \sigma(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{z})) \quad (2.18)$$

trong đó $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$ và $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$. Sigmoid cuối cho ra trọng số trong $(0, 1)$. Feature map được tái cân bằng:

$$\mathbf{F}'_c = m_c \cdot \mathbf{F}_c \quad (2.19)$$

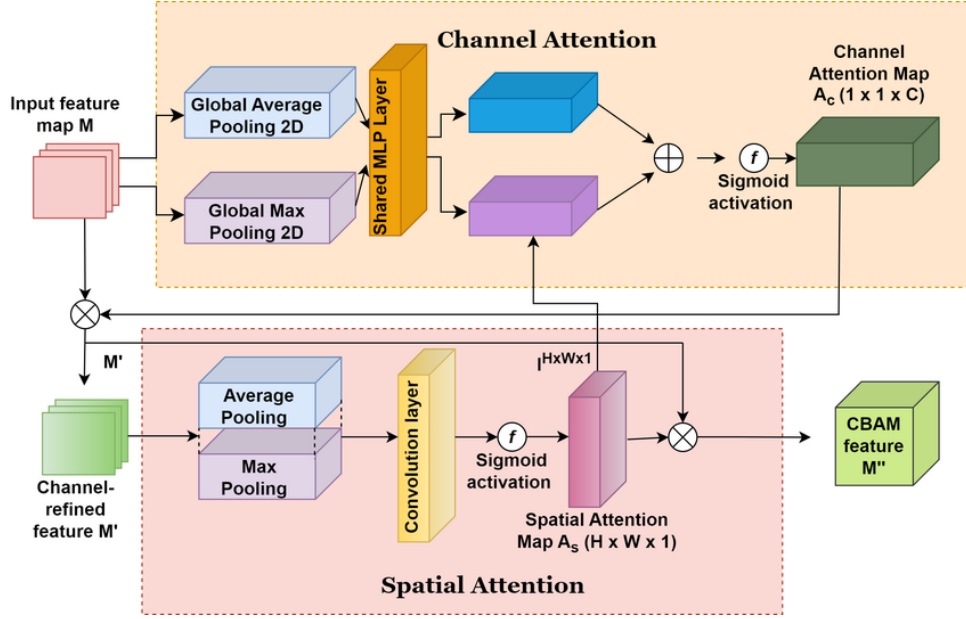
SE block có thể chèn vào bất kỳ vị trí nào trong mạng với chi phí tham số tối thiểu ($\approx 2C^2/r$, không đáng kể so với tổng tham số mạng). Hạn chế của SE là chỉ dùng GAP để nén không gian, có thể mất thông tin cục bộ và không có cơ chế tái cân bằng không gian.



Hình 2.10: Cấu trúc khối Squeeze-and-Excitation (SE Block): Squeeze nén thông tin không gian qua GAP, Excitation học trọng số kênh qua MLP hai lớp.

2.5.3 CBAM (Convolutional Block Attention Module)

CBAM [25] mở rộng SE theo hai hướng: cải tiến channel attention bằng cách dùng cả AvgPool lẫn MaxPool để tóm tắt thông tin phong phú hơn; và bổ sung spatial attention để tái cân bằng không gian sau khi tái cân bằng kênh. Hai loại attention được áp dụng tuần tự, mỗi loại tinh chỉnh lại đặc trưng trước khi đưa vào loại kia.



Hình 2.11: Cấu trúc CBAM: kết hợp tuần tự Channel Attention Module và Spatial Attention Module để tinh chỉnh feature map theo cả hai chiều.

Channel attention trong CBAM kết hợp song song AvgPool và MaxPool (nhằm bắt cả đặc trưng trung bình lẫn đặc trưng nổi bật nhất), rồi cộng kết quả của hai nhánh MLP chia sẻ trọng số:

$$\mathbf{M}_c(\mathbf{F}) = \sigma(\text{MLP}(\text{AvgPool}(\mathbf{F})) + \text{MLP}(\text{MaxPool}(\mathbf{F}))) \quad (2.20)$$

Spatial attention trong CBAM tổng hợp thông tin theo chiều kênh tại từng vị trí không gian bằng channel-wise AvgPool và MaxPool, ghép thành tensor $2 \times H \times W$ rồi áp dụng tích chập 7×7 để tạo bản đồ không gian:

$$\mathbf{M}_s(\mathbf{F}') = \sigma(f^{7 \times 7}[\text{AvgPool}_{ch}(\mathbf{F}'); \text{MaxPool}_{ch}(\mathbf{F}')]) \quad (2.21)$$

Pipeline tái cân bằng tuần tự của CBAM:

$$\mathbf{F}' = \mathbf{M}_c(\mathbf{F}) \otimes \mathbf{F}, \quad \mathbf{F}'' = \mathbf{M}_s(\mathbf{F}') \otimes \mathbf{F}' \quad (2.22)$$

Ý nghĩa trong bài toán KWS: Spatial attention tập trung vào dải tần số giọng nói (300–3400 Hz) và khoảng thời gian chứa âm vị, lọc bỏ nhiễu nền; trong khi channel

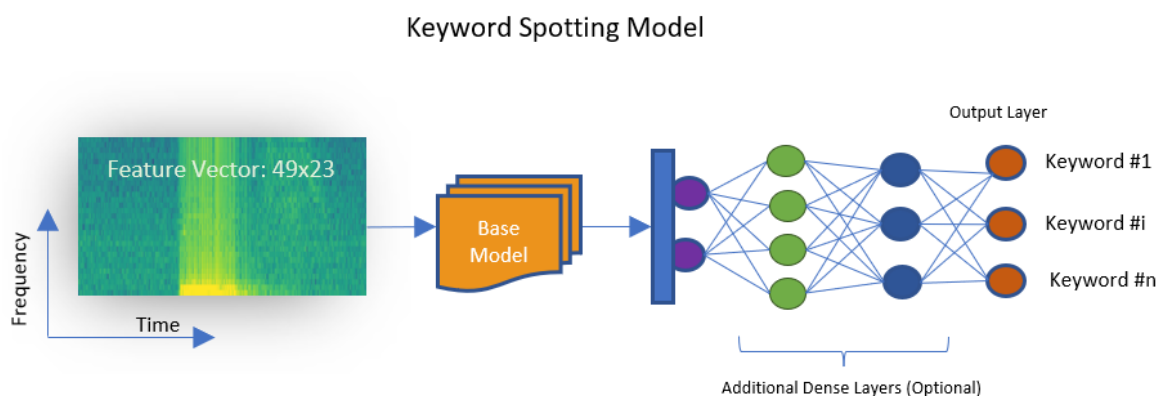
attention chọn lọc các bộ phát hiện đặc trưng (formant, nhịp điệu, cấu trúc hài hoà) phù hợp với từ khóa cần nhận dạng. Sự kết hợp tuần tự đảm bảo mỗi chiều chú ý được hưởng lợi từ kết quả của chiều kia. Nhờ khả năng chất lọc và tái cân bằng thông tin này, các kiến trúc tích hợp cơ chế chú ý (như CBAM) giúp mạng nơ-ron nâng cao đáng kể năng lực phân biệt từ khóa, duy trì độ chính xác cao ngay cả khi triển khai trong các môi trường có mức độ nhiễu nền phức tạp.

2.6 Nhận dạng từ khóa (Keyword Spotting)

2.6.1 Định nghĩa và đặc tả bài toán

Nhận dạng từ khóa (Keyword Spotting – KWS) là bài toán phân loại âm thanh nhằm phát hiện sự hiện diện của một tập từ hoặc cụm từ được định nghĩa trước trong luồng âm thanh liên tục, thành phần thiết yếu trong các hệ thống điều khiển bằng giọng nói cho IoT, nhà thông minh, thiết bị đeo tay, và xe hơi [3].

Về mặt toán học, KWS là bài toán phân loại đa lớp: cho đoạn âm thanh $\mathbf{x} \in \mathbb{R}^{f_s \cdot T}$ có độ dài T giây (thường $T = 1$), xác định nhãn $y \in \{1, \dots, K\}$ trong đó K là số từ khóa cộng thêm các lớp “unknown” và “silence”.



Hình 2.12: Cấu trúc mô hình KWS: trích xuất đặc trưng phổ tần từ tín hiệu âm thanh và đưa qua các lớp phân loại để đưa ra dự đoán từ khóa.

Phân biệt với ASR tổng quát: Nhận dạng giọng nói tổng quát (ASR) xử lý âm thanh tùy ý và tạo bản ghi văn bản; hệ thống ASR hiện đại như Whisper có hàng chục đến

hàng trăm triệu tham số và yêu cầu nhiều GB bộ nhớ. Ngược lại, KWS chỉ cần xác định sự hiện diện của từ trong tập cố định trong khung 1 giây và phải chạy liên tục, luôn bật (always-on). Trong triển khai thực tế, KWS dùng cửa sổ trượt (10–30 ms/bước) liên tục phân loại đoạn âm thanh hiện tại, đặt ra yêu cầu chặt chẽ về độ trễ và hiệu quả năng lượng.

2.6.2 Tập dữ liệu Google Speech Commands V2

Google Speech Commands V2 [9] là tập dữ liệu chuẩn phổ biến nhất cho KWS, gồm 105.829 đoạn WAV 1 giây (mono, 16 kHz, 16-bit PCM) thuộc 35 lớp từ khóa tiếng Anh, được thu bởi hơn 2.618 người dùng tình nguyện trong điều kiện môi trường đa dạng (không chỉ phòng thu chuẩn mà còn nhà bếp, văn phòng, ngoài trời). Phân chia chuẩn: 80% huấn luyện, 10% kiểm chứng, 10% kiểm tra.

35 lớp từ khóa bao gồm 10 từ lệnh chính (“yes”, “no”, “up”, “down”, “left”, “right”, “on”, “off”, “stop”, “go”), các chữ số 0–9, và các từ phụ trợ khác. Nghiên cứu này sử dụng cấu hình 35-class làm benchmark chính để đánh giá toàn diện hơn so với cấu hình 12-class phổ biến.

2.6.3 Thách thức triển khai trên thiết bị biên

Thiết bị biên đặt ra ba ràng buộc kỹ thuật phải đáp ứng đồng thời:

Ràng buộc bộ nhớ: Vi điều khiển ARM Cortex-M4 điển hình chỉ có 256 KB SRAM và 1 MB Flash, chỉ đủ cho mô hình vài trăm KB. Thiết bị cấp cao hơn như Raspberry Pi 4 có DRAM vài GB nhưng năng lượng hạn chế. Trong khi đó, AST [26] cần hơn 300 MB bộ nhớ – hoàn toàn không khả thi cho phần lớn phần cứng nhúng.

Ngân sách điện năng: Thiết bị IoT hoạt động bằng pin đòi hỏi tối ưu năng lượng mỗi lần suy luận E_{inf} (Joule/inference), không chỉ FLOPs lý thuyết. Hệ thống KWS luôn bật cần duy trì mức tiêu thụ điện ổn định và thấp ở tất cả các mức: vi điều khiển (<1 mW), SoC tầm trung (<100 mW), thiết bị đeo tay (<500 mW).

Ràng buộc độ trễ thời gian thực: Hệ thống KWS phải xử lý âm thanh nhanh hơn

tốc độ phát sinh thực tế, đánh giá qua Hệ số Thời gian Thực:

$$RTF = \frac{\text{Thời gian xử lý (s)}}{\text{Độ dài âm thanh (s)}} \quad (2.23)$$

Hệ thống đáp ứng thời gian thực khi $RTF < 1.0$; trong thực tế $RTF < 0.1$ cần thiết để dành đủ thời gian CPU cho các tác vụ khác trên cùng nền tảng.

2.6.4 Các kiến trúc KWS tiêu biểu

Bảng 2.1: So sánh các kiến trúc KWS tiêu biểu trên Google Speech Commands V2 (35 lớp)

Mô hình	Đặc điểm kiến trúc	Ưu điểm	Hạn chế
DS-CNN [3]	DSC 2D trên log-Mel	Nhẹ, nền tảng, dễ triển khai	Độ chính xác giới hạn ($\approx 95\%$)
MatchboxNet [27]	Conv 1D thời gian, sub-block	Cực nhỏ ($<100K$ params)	Mất tương quan tần số 2D
TC-ResNet [28]	ResNet + Conv 1D dọc thời gian	Hiệu quả tính toán, skip conn	Nhạy nhiều nền
MobileNetV2 [23]	Inverted Residual Block	FLOPs thấp, tốt cho CPU	Power Paradox, cần NAS tuning
KWT [29]	Transformer (patch embed-ding)	Độ chính xác cao ($\approx 98\%$)	Quá nặng cho edge ($>5M$ params)
AST [26]	ViT + pretrained ImageNet	SOTA accuracy (98.6%)	85M params, 325 MB, không khả thi cho edge

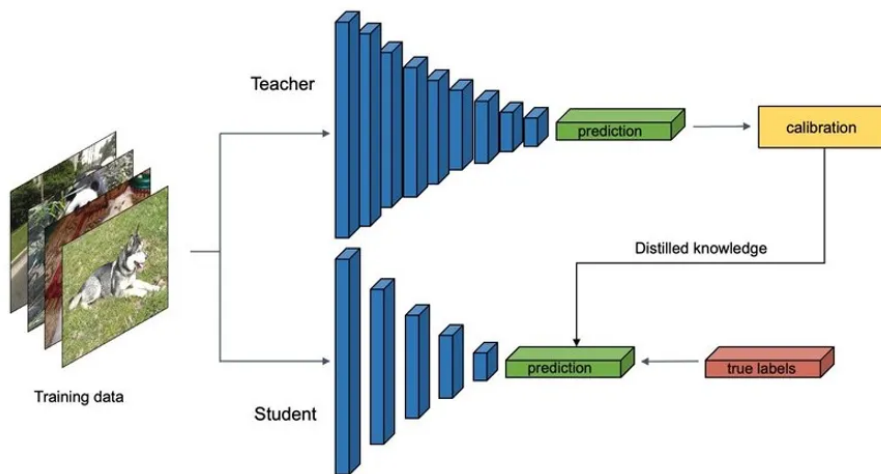
DS-CNN [3] là mô hình nền tảng cho KWS nhẹ, xây dựng từ các khối DSC của MobileNet, chứng minh DSC hoạt động hiệu quả trên log-Mel spectrogram với bốn phiên bản từ 24K đến 424K tham số. **KWT** [29] ứng dụng Vision Transformer đạt độ

chính xác rất cao nhưng số tham số (607K–5.4M) và chi phí tính toán vượt quá khả năng phần lớn thiết bị nhúng. Xu hướng nghiên cứu hiện tại tập trung vào *kết hợp CNN nhẹ với cơ chế chú ý* để tăng độ chính xác trong khi vẫn đáp ứng ràng buộc tài nguyên, đồng thời khai thác Chứng cất Tri thức để chuyển giao tri thức từ mô hình lớn (AST, KWT) sang mô hình nhỏ triển khai trực tiếp trên thiết bị.

2.7 Chứng cất tri thức (Knowledge Distillation)

2.7.1 Kiến trúc Giáo viên – Học sinh

Chứng cất tri thức (Knowledge Distillation – KD) [30] là kỹ thuật huấn luyện trong đó mô hình lớn đã được huấn luyện đầy đủ – *Giáo viên* (Teacher) – hướng dẫn quá trình huấn luyện của mô hình nhỏ hơn – *Học sinh* (Student). Teacher bị loại bỏ hoàn toàn sau huấn luyện; chỉ Student được triển khai, không làm tăng chi phí suy luận.



Hình 2.13: Cơ chế Chứng cất tri thức: Student được huấn luyện dựa trên cả nhãn thực tế (hard labels) và phân phối xác suất từ Teacher (soft labels).

Tri thức ẩn trong phân phối xác suất: Nhãn one-hot chỉ mang thông tin “đúng/sai” – rất thô. Ngược lại, phân phối xác suất đầu ra P_T của Teacher mang thông tin phong phú hơn: nó phản ánh mức độ tương đồng mà Teacher nhận thức được giữa các lớp. Ví dụ, khi nghe từ “four”, Teacher có thể cho xác suất “four” = 0.8, “for” = 0.1, “floor” = 0.05 – mã hoá thực tế rằng các từ này có phát âm gần nhau. Thông tin tương quan liên

lớp này không có trong nhãn one-hot nhưng rất có giá trị để Student học ranh giới quyết định tốt hơn.

Trong bài toán KWS, Teacher thường là AST (85M tham số, 98.6% accuracy), còn Student là mô hình siêu nhẹ thiết kế cho phần cứng nhúng. Trong quá trình huấn luyện, Teacher cung cấp nhãn mềm (soft labels) bổ sung cho nhãn cứng (hard labels) từ tập dữ liệu.

2.7.2 Hàm mất mát trong Knowledge Distillation

Temperature Scaling: Phân phối softmax thông thường ($T = 1$) thường rất “nhọn” – xác suất lớp đúng rất cao và các lớp khác gần 0 – khiến Teacher không khác nhãn one-hot nhiều. Hinton et al. [30] đề xuất tham số nhiệt độ $T > 1$ để làm phẳng phân phối:

$$P_T(z_i, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (2.24)$$

Khi $T > 1$ (thường $T \in [2, 10]$), các xác suất nhỏ được khuếch đại tương đối, lộ rõ mối quan hệ tương đồng liên lớp ẩn trong Teacher. T chỉ dùng khi tính soft loss trong huấn luyện; tại suy luận dùng $T = 1$.

Hàm mất mát tổng hợp: Kết hợp tuyến tính hard loss (cross-entropy với nhãn one-hot) và soft loss (KL divergence giữa phân phối mềm của Teacher và Student):

$$\mathcal{L}_{KD} = (1 - \alpha) \cdot \mathcal{L}_{hard} + \alpha \cdot T^2 \cdot D_{KL}(P_T^T \parallel P_S^T) \quad (2.25)$$

Hệ số T^2 bù lại sự co của gradient do temperature scaling, đảm bảo soft loss cân bằng với hard loss bất kể giá trị T . Tham số $\alpha \in [0, 1]$ điều chỉnh mức độ Student học từ Teacher so với nhãn thực; thực nghiệm cho thấy $\alpha = 0.7-0.9$ thường cho kết quả tốt nhất [30].

2.7.3 Chưng cất liên kiến trúc (Cross-Architecture KD)

Khi Teacher (Transformer) và Student (CNN) có kiến trúc hoàn toàn khác nhau, *khoảng cách năng lực biểu diễn* (capacity gap) tạo ra một thách thức cơ bản. Các mô hình Transformer như AST phân tích dữ liệu dựa trên cơ chế tự chú ý (self-attention)

toàn cục, trong khi mạng CNN siêu nhẹ lại giới hạn ở các vùng thụ cảm cục bộ. Do sự chênh lệch lớn về độ phức tạp cấu trúc này, Student có thể gặp khó khăn trong việc bắt chước không gian biểu diễn của Teacher [31].

Truyền đạt tri thức qua Logit mềm: Thay vì sử dụng các phương pháp chưng cất đặc trưng trung gian phức tạp (đòi hỏi phải đồng bộ hóa kích thước feature map giữa hai kiến trúc khác biệt), hướng tiếp cận hiệu quả và ít tốn kém hơn là tận dụng trực tiếp phân phối xác suất đầu ra (soft logits). Phân phối này chứa đựng "tri thức ẩn" (dark knowledge) mã hóa mối tương quan ngữ nghĩa giữa các lớp từ khóa. Bằng cách đối sánh KL Divergence trên các logit mềm, Student có thể học được ranh giới quyết định tinh vi của Teacher mà không bị gò bó bởi sự khác biệt về mặt cấu trúc vi mô.

Vai trò của cơ chế chú ý (Attention) trong kiến trúc Student: Để bù đắp lại giới hạn về năng lực biểu diễn khi chưng cất chéo kiến trúc, cơ chế chú ý kênh (CAM) bên trong mạng CNN Student đóng vai trò như một *bộ dịch đặc trưng*. Bằng cách tự động đánh giá lại tầm quan trọng của từng kênh và ức chế các kênh kích hoạt bởi nhiễu nền, CAM giúp Student hiệu quả hơn trong việc đồng hóa tri thức từ Teacher Transformer. Nói cách khác, CAM cung cấp cho mô hình CNN nhỏ một công cụ chắt lọc thông tin tương tự như self-attention, giúp nó tập trung chính xác vào tín hiệu giọng nói mục tiêu để hấp thụ tối đa lượng tri thức truyền tải qua các logit mềm.

2.8 Các phương pháp và thang đo đánh giá

2.8.1 Đánh giá độ chính xác phân loại

Độ chính xác Top-1: Tỷ lệ mẫu có lớp dự đoán xác suất cao nhất trùng với nhãn thực:

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i] \quad (2.26)$$

Đây là thang đo chính trong benchmark KWS 35 lớp, dùng để so sánh trực tiếp các mô hình.

AUC: Bên cạnh accuracy, *Area Under the ROC Curve* (AUC) được sử dụng làm chỉ số đánh giá bổ sung, phản ánh khả năng phân biệt tổng quát của mô hình trên toàn bộ các

ngưỡng quyết định thay vì chỉ tại một ngưỡng cố định. AUC đo diện tích dưới đường cong ROC (Receiver Operating Characteristic), biểu diễn quan hệ giữa True Positive Rate và False Positive Rate khi ngưỡng phân loại thay đổi. Đối với bài toán phân loại đa lớp như KWS, AUC được tính bằng cách tổng hợp các so sánh theo cặp lớp (pairwise) [32]. Giá trị AUC càng gần 1 cho thấy mô hình càng có khả năng phân tách tốt giữa các lớp từ khóa, trong khi $AUC = 0.5$ tương ứng với một bộ phân loại ngẫu nhiên.

2.8.2 Đánh giá độ phức tạp và hiệu năng phần cứng

Số tham số và MACs: Số tham số quyết định dung lượng bộ nhớ cần thiết (1 tham số float32 = 4 byte; int8 = 1 byte sau lượng tử hoá). MACs là đơn vị thực tế hơn FLOPs cho phần cứng neural (1 MAC = 1 nhân + 1 cộng = 2 FLOPs). Lưu ý quan trọng: FLOPs thấp không đảm bảo tiêu thụ năng lượng thấp trên phần cứng thực (xem mục 2.4.4).

RTF và Throughput: RTF (phương trình 2.23) là điều kiện cần cho hệ thống thời gian thực. Throughput (IPS – Inferences Per Second) đo với batch size = 1 để mô phỏng hệ thống luồng đơn trên thiết bị biên.

Năng lượng mỗi suy luận E_{inf} : Chỉ số tổng hợp phản ánh chính xác nhất hiệu quả năng lượng trong ứng dụng KWS luân bật (phương trình 2.16), cho phép so sánh công bằng giữa các mô hình có throughput khác nhau. Điện năng trung bình P_{avg} được đo bằng thiết bị đo năng lượng phần cứng trong quá trình suy luận.

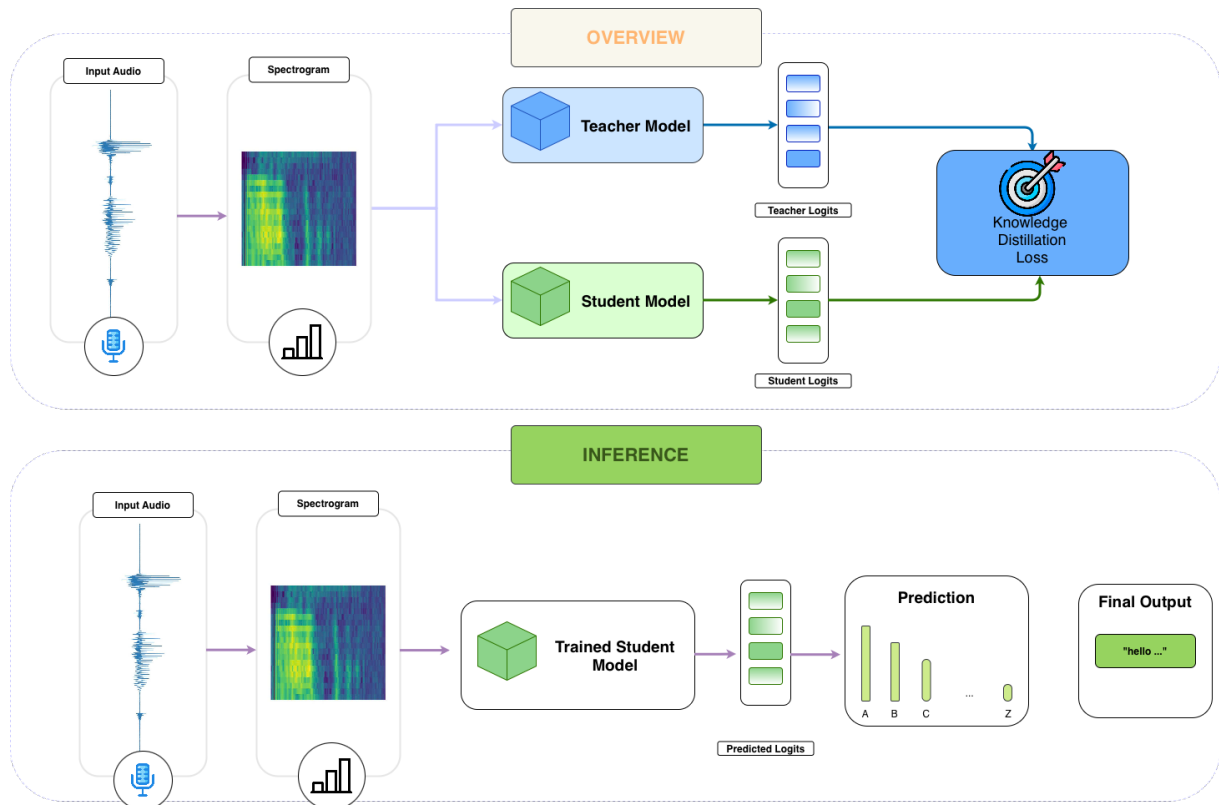
Chương 3

THIẾT KẾ HỆ THỐNG

3.1 Tổng quan hệ thống

3.1.1 Kiến trúc tổng thể

Hệ thống được đề xuất trong nghiên cứu này được xây dựng theo kiến trúc Teacher–Student, hoạt động qua hai pha tách biệt: **pha huấn luyện** (training) và **pha suy luận** (inference). Hình 3.1 minh họa tổng quan pipeline của toàn bộ hệ thống.



Hình 3.1: Kiến trúc tổng thể của hệ thống.

Pha huấn luyện (Training)

Trong pha này, các mẫu âm thanh từ tập dữ liệu huấn luyện được tiền xử lý thành Log-Mel Spectrogram và đưa *đồng thời* vào hai mô hình song song. Nhánh thứ nhất là mô hình Teacher (AST – Audio Spectrogram Transformer), đã được huấn luyện trước (pre-trained) và **cố định toàn bộ trọng số** trong suốt quá trình này; Teacher không được cập nhật mà chỉ đóng vai trò nguồn cung cấp *nhãn mềm* (soft labels) dưới dạng phân phối xác suất qua các Teacher Logits. Nhánh thứ hai là mô hình Student (QANet-Lite), đây là mô hình nhẹ được đề xuất và là đối tượng huấn luyện chính; Student tự sinh ra Student Logits từ cùng đầu vào spectrogram đó. Hàm mất mát tổng hợp được tính từ hai nguồn tín hiệu giám sát:

- **Hard Loss:** Cross-entropy tiêu chuẩn giữa Student Logits và nhãn cứng (ground-truth one-hot labels) từ tập dữ liệu.
- **Soft Loss:** KL Divergence giữa phân phối xác suất có nhiệt độ T của Teacher và của Student, mang thông tin tương quan liên lớp mà Teacher đã học được.

Tổ hợp hai thành phần này tạo thành *Knowledge Distillation Loss*, được dùng để cập nhật trọng số của Student thông qua lan truyền ngược.

Pha suy luận (Inference)

Sau khi hoàn tất huấn luyện, mô hình Teacher được **loại bỏ hoàn toàn** và không tham gia vào bất kỳ bước xử lý nào về sau. Chỉ có QANet-Lite đã được huấn luyện được triển khai lên thiết bị biên. Tín hiệu âm thanh thô tiếp tục được chuyển đổi thành Log-Mel Spectrogram rồi đưa qua Student để sinh ra Predicted Logits; lớp có xác suất cao nhất được chọn làm kết quả phân loại cuối cùng (ví dụ: "hello"). Thiết kế này đảm bảo tri thức được chưng cất từ Teacher trong quá trình huấn luyện không phát sinh thêm bất kỳ chi phí tính toán hay bộ nhớ nào tại thời điểm suy luận trên thiết bị thực tế.

3.1.2 Tiền xử lý đầu vào

Trích xuất đặc trưng âm thanh. Trước khi đưa vào mạng nơ-ron, tín hiệu âm thanh một chiều được chuyển đổi thành đặc trưng âm học 2D. Mỗi đoạn âm thanh tính dài 1 giây (lấy mẫu tại 16 kHz) được biến đổi thành Log-Mel Spectrogram thông qua phép biến đổi Short-Time Fourier Transform (STFT) với các thông số chi tiết trong Bảng 3.1. Với bước trượt cửa sổ 160 mẫu (10 ms), một đoạn âm thanh 1 giây ban đầu tạo ra khoảng 100 khung thời gian. Tín hiệu sau đó được điền không (zero-padding) để mở rộng trục thời gian, tạo ra tensor đặc trưng đồng nhất có kích thước 128×128 , tối ưu cho việc tính toán của mạng CNN.

Để ổn định quá trình huấn luyện và loại bỏ sự chênh lệch về biên độ âm lượng giữa các người nói, các tensor ảnh phổ được chuẩn hoá Z-score theo thống kê toàn cục của tập dữ liệu huấn luyện, với giá trị trung bình $\mu = -6.8459$ và độ lệch chuẩn $\sigma = 5.5654$ [26].

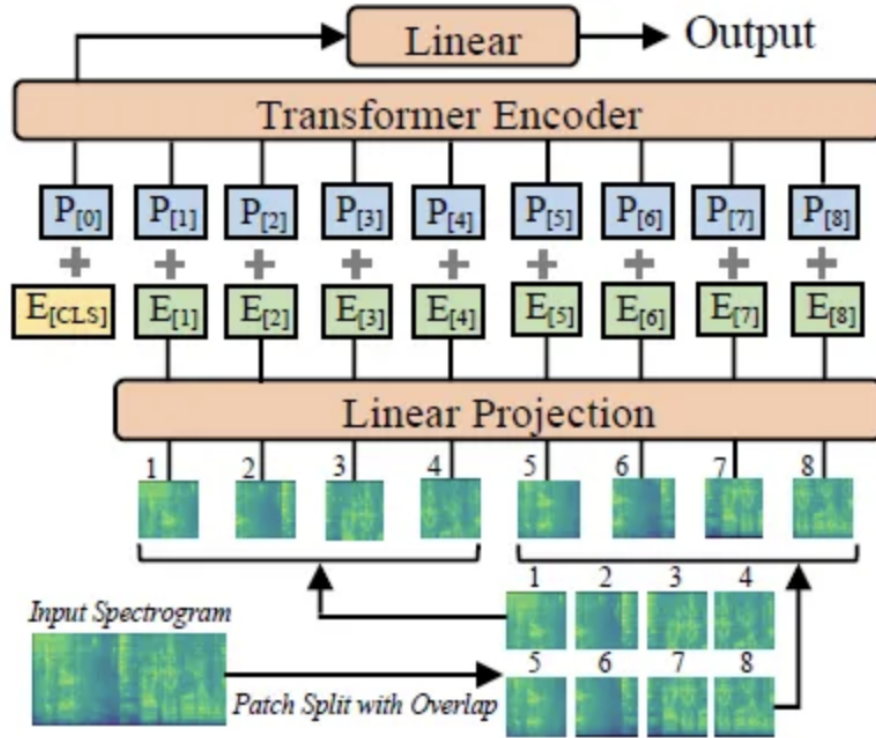
Bảng 3.1: Thông số cấu hình trích xuất Log-Mel Spectrogram

Thông số	Giá trị	Mô tả
Sample rate	16 000 Hz	Tần số lấy mẫu đầu vào
Clip duration	1 giây	Độ dài mỗi đoạn âm thanh
FFT size (N)	512	Độ dài cửa sổ STFT
Hop length (H)	160	Bước trượt cửa sổ (10 ms)
Window function	Hann	Hàm cửa sổ giảm rò rỉ phổ
Mel filters (M)	128	Số lượng băng lọc Mel
Freq range	0–8000 Hz	Dải tần số tương ứng định lý Nyquist
Output size	128×128	Tensor đặc trưng (Thời gian $\times$ Tần số)

3.2 Mô hình Giáo viên (Teacher Model)

3.2.1 Audio Spectrogram Transformer

Nghiên cứu này sử dụng Audio Spectrogram Transformer (AST) [26] làm mô hình Teacher. AST là kiến trúc dựa trên Vision Transformer (ViT) được điều chỉnh cho dữ liệu âm thanh, đạt độ chính xác 98.17% trên bộ dữ liệu Google Speech Commands V2. Mô hình tách Log-Mel Spectrogram thành các patch 16×16 , mã hoá vị trí và xử lý qua 12 lớp self-attention. Nhờ cơ chế self-attention toàn cục, AST nắm bắt được mối quan hệ dài hạn giữa các khung thời gian và dải tần số – một khả năng mà CNN với trường thụ cảm hữu hạn không có được. Chính biểu diễn đặc trưng toàn cục này là nguồn tri thức giá trị cần được chưng cất sang mô hình Student.



AST: Audio Spectrogram Transformer

Hình 3.2: Kiến trúc mô hình Teacher (AST).

3.2.2 Vai trò và giới hạn triển khai

Mặc dù đạt độ chính xác vượt trội, AST với 85,4 triệu tham số (325,81 MB) hoàn toàn không phù hợp để triển khai trực tiếp trên thiết bị biên [1, 3]. Trong pipeline đề xuất, AST được sử dụng hoàn toàn offline trong pha huấn luyện: trọng số được đóng băng, mô hình chỉ thực hiện forward pass để tạo logits mềm $\mathbf{z}_T$ cho mỗi batch. Sau khi huấn luyện Student hoàn tất, Teacher không được đóng gói hay triển khai cùng hệ thống – chi phí suy luận của hệ thống cuối chỉ phụ thuộc vào kích thước của Student.

3.3 Kiến trúc mô hình học sinh đề xuất (QANet-Lite)

3.3.1 Tổng quan thiết kế

Động lực và triết lý thiết kế

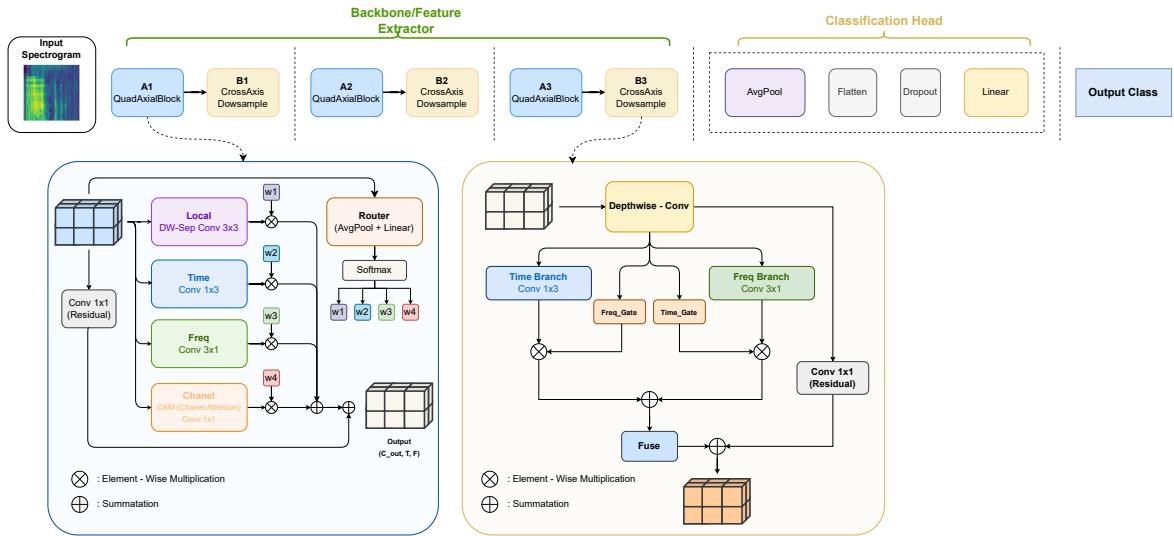
Bài toán nhận dạng từ khóa trên thiết bị biên đặt ra hai yêu cầu mâu thuẫn nhau: mô hình phải đủ nhỏ để vừa với bộ nhớ hạn chế của vi điều khiển, nhưng đồng thời phải đủ mạnh để xử lý Log-Mel Spectrogram – một biểu diễn 2D phức tạp với cấu trúc ngữ nghĩa đan xen trên cả hai trục thời gian và tần số. Các kiến trúc CNN nhẹ hiện có như DS-CNN [3] hay MobileNetV2 [23] được thiết kế cho ảnh tự nhiên, xử lý đồng đều cả hai trục không gian – điều này không tối ưu cho spectrogram vì trục thời gian và trục tần số mang ngữ nghĩa hoàn toàn khác nhau và cần được phân tích chuyên biệt.

Từ những hạn chế đó, QANet-Lite(Quad-Axial Network Lite) được đề xuất với triết lý thiết kế phân tích đa trục có định tuyến động (dynamic-routed multi-axis analysis): thay vì xử lý spectrogram như một ảnh thông thường, mô hình chủ động phân tách việc phân tích theo trục thời gian, trục tần số, không gian cục bộ, và trục kênh thành bốn luồng độc lập song song, sau đó dùng cơ chế định tuyến học được để tổng hợp thích nghi tùy theo đặc điểm âm học của từng mẫu đầu vào.

Tên **QANet-Lite** phản ánh đầy đủ ba đặc trưng cốt lõi của kiến trúc: *Quad* – bốn nhánh phân tích song song; *Axial* – phân tích chuyên biệt theo từng trục của spectrogram; và *Lite* – thiết kế nhẹ hướng đến thiết bị biên với ràng buộc tài nguyên nghiêm ngặt.

Cấu trúc tổng thể

QANet-Lite tổ chức theo kiến trúc tháp ba giai đoạn (three-stage pyramid), trong đó mỗi giai đoạn gồm hai khối liên tiếp: khối trích xuất đặc trưng đa nhánh (**QuadAxialBlock**) và khối giảm mẫu lai có cơ chế cổng thông tin chéo (**CrossAxisDownsample**). Qua mỗi giai đoạn, số kênh tăng gấp đôi trong khi kích thước không gian giảm một nửa – cơ chế quen thuộc trong các mạng CNN hiện đại giúp mô hình học được đặc trưng từ cục bộ đến toàn cục theo từng tầng trừu tượng.



Hình 3.3: Kiến trúc tổng quan của QANet-Lite gồm ba giai đoạn, mỗi giai đoạn bao gồm khối QuadAxialBlock (A) và CrossAxisDownsample (B). Chi tiết QuadAxialBlock (dưới trái) với bốn nhánh song song và bộ định tuyến động; chi tiết CrossAxisDownsample (dưới phải) với cơ chế cổng chéo theo trục thời gian–tần số.

Luồng dữ liệu qua mô hình được tóm tắt như sau:

$$(1, 128, 128) \xrightarrow{\text{Stage 1}} (C_1, 64, 64) \xrightarrow{\text{Stage 2}} (C_2, 32, 32) \xrightarrow{\text{Stage 3}} (C_3, 16, 16) \xrightarrow{\text{Head}} (K = 35) \quad (3.1)$$

Đầu vào là Log-Mel Spectrogram kích thước $1 \times 128 \times 128$ (1 kênh đơn sắc, 128 khung thời gian, 128 băng Mel). Sau ba giai đoạn, feature map $C_3 \times 16 \times 16$ được thu gọn qua Classification Head thành vector logit 35 chiều tương ứng 35 lớp từ khóa.

Thiết kế thấp đặc trưng này có ý nghĩa ngữ nghĩa rõ ràng trong bài toán KWS: Stage 1 (độ phân giải 128×128) học các đặc trưng cục bộ có tính chi tiết cao – ranh giới âm vị, chuyển tiếp formant, các biến đổi năng lượng tức thời; Stage 2 (64×64) tổng hợp các đặc trưng ở mức độ âm tiết – cấu trúc nguyên âm-phụ âm, mẫu nhịp điệu; Stage 3 (32×32) nắm bắt cấu trúc từ khóa toàn cục – dạng bao năng lượng, cấu trúc hài hoà mở rộng. Sự phân tầng này cho phép mô hình nhỏ gọn nắm bắt thông tin đa tầng mà không cần tăng chiều sâu mạng.

Bảng 3.2 trình bày chi tiết cấu hình từng lớp:

Bảng 3.2: Cấu trúc topo chi tiết của QANet-Lite

Giai đoạn	Module	Cấu hình	Output shape
Input	–	Log-Mel Spectrogram	$1 \times 128 \times 128$
Stage 1	QuadAxialBlock	in=1, out= C_1	$C_1 \times 128 \times 128$
	CrossAxisDownsample	in= C_1 , out= C_1	$C_1 \times 64 \times 64$
Stage 2	QuadAxialBlock	in= C_1 , out= C_2	$C_2 \times 64 \times 64$
	CrossAxisDownsample	in= C_2 , out= C_2	$C_2 \times 32 \times 32$
Stage 3	QuadAxialBlock	in= C_2 , out= C_3	$C_3 \times 32 \times 32$
	CrossAxisDownsample	in= C_3 , out= C_3	$C_3 \times 16 \times 16$
Head	AvgPool	Global Average Pooling	$C_3 \times 1 \times 1$
	Flatten	–	C_3
	Dropout	–	C_3
	Linear	$C_3 \rightarrow 35$	35

3.3.2 Mô-đun chú ý kênh (Channel Attention Module – CAM)

Động lực và vị trí trong kiến trúc

CAM là thành phần nền tảng được tái sử dụng như một nhánh xử lý độc lập bên trong QuadAxialBlock. Mục tiêu của CAM là trả lời câu hỏi “*kênh đặc trưng nào quan trọng cho việc nhận dạng từ khóa hiện tại?*” – bổ sung cho ba nhánh không gian chuyên phân tích ở đâu và như thế nào thông tin xuất hiện trên spectrogram.

CAM kế thừa cơ chế Squeeze-and-Excitation của SENet [24] nhưng mở rộng ở bước tổng hợp toàn cục: thay vì chỉ dùng Global Average Pooling (GAP) như SE gốc, CAM sử dụng đồng thời cả GAP lẫn Global Max Pooling (GMP). Lý do là hai phép pooling này bổ sung nhau về ngữ nghĩa: GAP phản ánh mức kích hoạt trung bình – đặc trưng phân phối năng lượng tổng thể của kênh trên toàn spectrogram; GMP nhạy hơn với các đỉnh kích hoạt cục bộ – đặc trưng sự hiện diện của mẫu đặc trưng nổi bật nhất trong kênh. Kết hợp cả hai cho phép CAM ra quyết định chú ý kênh dựa trên bức tranh đầy đủ hơn so với chỉ dùng một phép pooling.

Cơ chế hoạt động

Cho feature map đầu vào $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$, CAM thực hiện qua ba bước:

Bước 1 – Tổng hợp toàn cục kép: Toàn bộ chiều không gian $H \times W$ của mỗi kênh được nén thành hai véc-tơ C chiều thông qua GAP và GMP:

$$\mathbf{f}^{avg} = \text{GAP}(\mathbf{F}) \in \mathbb{R}^C, \quad \mathbf{f}^{max} = \text{GMP}(\mathbf{F}) \in \mathbb{R}^C \quad (3.2)$$

Mỗi phần tử f_c^{avg} và f_c^{max} mã hoá thông tin tóm tắt về kênh c từ hai góc nhìn bổ sung nhau. Bước này tương tự bước Squeeze trong SE – loại bỏ thông tin vị trí để tập trung vào *cái gì* hơn là *ở đâu*.

Bước 2 – Học trọng số kênh qua MLP chia sẻ trọng số: Cả hai véc-tơ được xử lý qua *cùng* một MLP hai lớp (hệ số nén $r = 4$, chiều ẩn C/r , kích hoạt ReLU ở giữa), rồi cộng kết quả và đưa qua sigmoid:

$$\mathbf{m}_c = \sigma(\text{MLP}(\mathbf{f}^{avg}) + \text{MLP}(\mathbf{f}^{max})) \quad (3.3)$$

Việc chia sẻ trọng số MLP cho cả hai nhánh pooling là thiết kế hiệu quả tham số: chỉ thêm $2 \cdot (C \cdot C/r + C/r \cdot C)$ tham số – với $r = 4$ tức chỉ $C^2/2$ tham số, không đáng kể so với toàn bộ mạng. Sigmoid đầu ra cho trọng số kênh $\mathbf{m}_c \in (0, 1)^C$, trong đó giá trị gần 1 biểu thị kênh quan trọng và giá trị gần 0 biểu thị kênh cần ức chế.

Bước 3 – Tái cân bằng đặc trưng: Trọng số kênh được áp dụng bằng phép nhân broadcast theo chiều kênh:

$$\mathbf{F}' = \mathbf{m}_c \otimes \mathbf{F} \quad (3.4)$$

Đây là bước Excitation – khuếch đại các kênh mang đặc trưng giọng nói quan trọng (formant, hài âm, cấu trúc phổ đặc trưng từ khóa) và ức chế các kênh bị kích hoạt bởi nhiễu nền hoặc đặc trưng không liên quan.

Sự khác biệt so với SE và CBAM

So với SE [24], CAM trong QANet-Lite có hai điểm khác biệt quan trọng. Thứ nhất, CAM dùng cả GAP lẫn GMP thay vì chỉ GAP, cung cấp thông tin chú ý kênh toàn diện

hơn. Thứ hai và quan trọng hơn, CAM không được áp dụng *tuần tự sau* tích chập chính như SE hay CBAM [25] – thay vào đó, CAM được tích hợp như một trong bốn nhánh *song song* trong QuadAxialBlock, cho phép thông tin chú ý kênh tương tác và cạnh tranh trực tiếp với thông tin không gian từ các nhánh khác thông qua bộ định tuyến động.

3.3.3 Khối trích xuất đặc trưng đa nhánh (QuadAxialBlock)

Ý tưởng thiết kế

QuadAxialBlock là khối xử lý trung tâm và đặc trưng nhất của QANet-Lite. Ý tưởng xuất phát từ một quan sát căn bản về Log-Mel Spectrogram: không gian $T \times F$ của spectrogram không đồng nhất – trục thời gian T và trục tần số F có ngữ nghĩa hoàn toàn khác nhau. Một bộ lọc vuông $k \times k$ thông thường xử lý cả hai trục như nhau, điều này không tận dụng được sự khác biệt ngữ nghĩa này. Hơn nữa, không phải từ khóa nào cũng có đặc trưng tương tự: từ khóa giàu nguyên âm như “go” hay “no” có cấu trúc tần số đặc trưng rõ rệt (formant ổn định); trong khi từ khóa giàu phụ âm bật như “stop” hay “left” lại có biến đổi đặc trưng theo thời gian mạnh hơn. Một kiến trúc cứng nhắc không thể thích nghi với sự đa dạng này.

QuadAxialBlock giải quyết cả hai vấn đề trên bằng cách: (1) tách biệt việc phân tích theo trục thời gian, trục tần số, không gian cục bộ, và kênh thành bốn luồng song song chuyên biệt; (2) học động trọng số tổng hợp tùy theo nội dung từng mẫu đầu vào thay vì dùng trọng số cố định.

Bốn nhánh song song

Đầu vào $\mathbf{F} \in \mathbb{R}^{C_{in} \times T \times F}$ được đưa đồng thời vào bốn nhánh, mỗi nhánh chuyên biệt hoá vào một khía cạnh riêng biệt:

Nhánh Local – Tích chập tách biệt chiều sâu 3×3 : Nhánh này trích xuất đặc trưng cục bộ trong vùng lân cận 3×3 trên cả hai chiều thời gian-tần số – nắm bắt các mẫu không gian 2D như ranh giới formant-nhiều, sự biến đổi đột ngột trong cả hai chiều cùng lúc. Để tiết kiệm tính toán, nhánh này dùng depthwise separable convolution thay vì tích chập tiêu chuẩn, giảm chi phí khoảng 8–9 lần [11] mà không mất đáng kể khả năng biểu

diễn:

$$\mathbf{F}_1 = \text{BN}(\text{ReLU}(\text{DW-SepConv}_{3 \times 3}(\mathbf{F}))) \quad (3.5)$$

Nhánh Time – Tích chập bất đối xứng 1×3 : Bộ lọc 1×3 (một hàng, ba cột) chỉ nhìn theo chiều ngang của spectrogram – tức chiều thời gian – mà không tác động lên chiều tần số. Nhánh này học cách phát hiện sự thay đổi âm học theo thời gian: chuyển tiếp giữa các âm vị (ví dụ từ phụ âm sang nguyên âm trong “stop”), mẫu biến đổi năng lượng, và cấu trúc nhịp điệu đặc trưng của từng từ khóa:

$$\mathbf{F}_2 = \text{BN}(\text{ReLU}(\text{Conv}_{1 \times 3}(\mathbf{F}))) \quad (3.6)$$

Nhánh Freq – Tích chập bất đối xứng 3×1 : Đối xứng với nhánh Time, bộ lọc 3×1 (ba hàng, một cột) chỉ nhìn theo chiều dọc – tức chiều tần số. Nhánh này chuyên nắm bắt cấu trúc hài hoà (harmonic structure) của giọng nói: mối quan hệ giữa tần số cơ bản và các hài âm, vị trí của các formant (F1, F2, F3) đặc trưng cho từng nguyên âm, và cấu trúc cộng hưởng phân biệt các âm vị khác nhau:

$$\mathbf{F}_3 = \text{BN}(\text{ReLU}(\text{Conv}_{3 \times 1}(\mathbf{F}))) \quad (3.7)$$

Nhánh Channel – CAM kết hợp tích chập 1×1 : Khác với ba nhánh không gian phân tích *cấu trúc vị trí* trong spectrogram, nhánh Channel phân tích *tầm quan trọng tương đối của các kênh đặc trưng*. CAM (được mô tả ở mục trước) học cách chọn lọc kênh nào mang thông tin hữu ích cho từ khóa hiện tại – về bản chất là đặt câu hỏi “detector đặc trưng nào (trong số C_{in} detector) đang nhận diện thứ gì đó có nghĩa?” Tích chập 1×1 sau đó chiếu thông tin đã được chú ý kênh sang không gian C_{out} kênh:

$$\mathbf{F}_4 = \text{BN}(\text{ReLU}(\text{Conv}_{1 \times 1}(\text{CAM}(\mathbf{F})))) \quad (3.8)$$

Bốn nhánh này khai thác bốn loại thông tin bổ sung, không chồng chéo nhau. Sự phân chia rõ ràng theo chức năng này là điểm mạnh thiết kế: mỗi nhánh được phép chuyên biệt hoá sâu vào một khía cạnh mà không bị “phân tâm” bởi các loại thông tin khác, trong khi bộ định tuyến động quyết định trọng số tổng hợp phù hợp.

Bộ định tuyến động (Dynamic Router)

Bộ định tuyến động là thành phần phân biệt QuadAxialBlock với các kiến trúc đa nhánh truyền thống như Inception [20] vốn kết hợp các nhánh với trọng số cố định (thường là ghép kênh đều nhau). Thay vào đó, QuadAxialBlock *học* tỷ lệ đóng góp phù hợp của từng nhánh dựa trên nội dung cụ thể của từng mẫu đầu vào.

Về cơ chế, bộ định tuyến tóm tắt ngữ cảnh toàn cục của feature map $\mathbf{F}$ bằng Global Average Pooling thành véc-tơ C_{in} chiều, rồi ánh xạ qua lớp tuyến tính và chuẩn hoá bằng Softmax để tạo ra bốn trọng số:

$$[w_1, w_2, w_3, w_4] = \text{Softmax}(\mathbf{W}_r \text{AvgPool}(\mathbf{F})) \quad (3.9)$$

trong đó $\mathbf{W}_r \in \mathbb{R}^{4 \times C_{in}}$ là tham số học được, chỉ thêm $4 \times C_{in}$ tham số – chi phí rất nhỏ. Ràng buộc Softmax đảm bảo $\sum_{i=1}^4 w_i = 1$: bộ định tuyến phân phối một “ngân sách chú ý” cố định, và nhánh nào quan trọng hơn với mẫu hiện tại sẽ nhận phần ngân sách lớn hơn.

Ý nghĩa thực tiễn của thiết kế này trong KWS rất rõ ràng. Xét hai từ khóa đối lập về đặc tính âm học:

- Từ “go” (nguyên âm /o/ kéo dài): bộ định tuyến dự kiến sẽ phân bổ trọng số cao hơn cho nhánh Freq (phát hiện cấu trúc formant ổn định) và nhánh Channel (chọn lọc kênh nhạy cảm với nguyên âm);
- Từ “stop” (phụ âm bật /st/ + nguyên âm ngắn): bộ định tuyến dự kiến ưu tiên nhánh Time (phát hiện chuyển tiếp đột ngột) và nhánh Local (nhắm bắt biến đổi không gian 2D tại điểm bật âm).

Khả năng thích nghi theo nội dung này giúp cùng một kiến trúc xử lý tốt tất cả 35 lớp từ khóa với đặc trưng âm học đa dạng, mà không cần tăng độ sâu hay số tham số.

Tổng hợp đầu ra và kết nối tắt

Đầu ra của bốn nhánh được tổng hợp có trọng số rồi cộng với kết nối tắt (residual) qua tích chập 1×1 để khớp chiều kênh:

$$\mathbf{F}_{out} = \sum_{i=1}^4 w_i \cdot \mathbf{F}_i + \text{Conv}_{1 \times 1}(\mathbf{F}) \quad (3.10)$$

Kết nối tắt $\text{Conv}_{1 \times 1}(\mathbf{F})$ đóng vai trò kép trong thiết kế này. Thứ nhất, nó tạo đường dẫn gradient trực tiếp qua khối – điều kiện cần để huấn luyện ổn định khi xếp chồng ba giai đoạn liên tiếp, theo tinh thần của ResNet [19]. Thứ hai, kết nối tắt cho phép mô hình học *phần dư* so với đầu vào: thay vì học toàn bộ biến đổi từ $\mathbf{F}$ sang $\mathbf{F}_{out}$, mô hình chỉ cần học “điều chỉnh bổ sung” so với đặc trưng ban đầu – nhiệm vụ tối ưu dễ hơn đáng kể.

3.3.4 Khối giảm mẫu lai (CrossAxisDownsample)

Động lực thiết kế

Sau mỗi QuadAxialBlock, mô hình cần giảm kích thước không gian của feature map để tăng vùng thụ cảm và giảm chi phí tính toán cho các stage tiếp theo. Cách tiếp cận truyền thống – Max Pooling hoặc Average Pooling – có nhược điểm cơ bản: chúng chỉ thực hiện lấy mẫu dưới (downsampling) một cách cứng nhắc dựa trên phép toán cố định, không học được cách chọn lọc thông tin phù hợp với tác vụ. Trong bài toán KWS, mỗi lần giảm mẫu đều làm mất thông tin không thể phục hồi; do đó, điều quan trọng là mô hình phải *học* được cách giảm mẫu thông minh – giữ lại thông tin quan trọng nhất cho việc nhận dạng từ khóa.

CrossAxisDownsample giải quyết vấn đề này bằng cách kết hợp giảm mẫu có học với cơ chế lọc thông tin chéo trục (cross-axis gating): thông tin từ trục tần số kiểm soát những gì được giữ lại từ trục thời gian và ngược lại. Cơ chế này buộc mô hình học biểu diễn liên trục – trong đó đặc trưng thời gian chỉ được giữ lại khi được xác nhận bởi thông tin tần số, và ngược lại.

Luồng xử lý bốn bước

Bước 1 – Giảm mẫu bằng Depthwise Convolution stride=2: Depthwise convolution 3×3 với bước nhảy 2 thực hiện đồng thời hai nhiệm vụ: lọc không gian để trích xuất đặc trưng, và giảm kích thước không gian từ $T \times F$ xuống $T/2 \times F/2$. Việc dùng DWConv thay vì tích chập tiêu chuẩn cho bước giảm mẫu là lựa chọn hợp lý vì chi phí tính toán thấp hơn đáng kể, và bước giảm mẫu không cần trao đổi thông tin giữa các kênh. Kết quả là feature map trung gian $\hat{\mathbf{F}} \in \mathbb{R}^{C \times T/2 \times F/2}$:

$$\hat{\mathbf{F}} = \text{BN}(\text{ReLU}(\text{DWConv}_{3 \times 3, s=2}(\mathbf{F}))) \quad (3.11)$$

Bước 2 – Phân tách theo hai trục song song: Từ feature map đã giảm mẫu $\hat{\mathbf{F}}$, hai nhánh song song tinh lọc thông tin theo từng trục chuyên biệt:

- *Time Branch* – tích chập 1×3 : tinh lọc theo chiều thời gian, học cách kết hợp thông tin từ ba khung thời gian liên tiếp sau giảm mẫu, tạo $\mathbf{F}_{time}$.
- *Freq Branch* – tích chập 3×1 : tinh lọc theo chiều tần số, học cách kết hợp thông tin từ ba băng tần liên tiếp sau giảm mẫu, tạo $\mathbf{F}_{freq}$.

Cả hai nhánh đều áp dụng BN và ReLU sau tích chập. Việc tinh lọc sau giảm mẫu (thay vì trước) đảm bảo mô hình học cách khai thác thông tin trong không gian đã được nén, phù hợp hơn với tác vụ phân tích ở độ phân giải thấp hơn.

Bước 3 – Cơ chế cổng chéo (Cross-Axis Gating): Đây là thành phần độc đáo nhất của CrossAxisDownsample. Thay vì chỉ đơn giản cộng hoặc ghép hai nhánh, mỗi nhánh được điều chỉnh bởi cổng mềm (soft gate) từ nhánh kia:

$$\mathbf{F}'_{time} = \mathbf{F}_{time} \odot \sigma(\mathbf{F}_{freq}) \quad (3.12)$$

$$\mathbf{F}'_{freq} = \mathbf{F}_{freq} \odot \sigma(\mathbf{F}_{time}) \quad (3.13)$$

Hàm sigmoid $\sigma(\cdot)$ biến đặc trưng từ nhánh kia thành bản đồ cổng trong khoảng $(0, 1)$. Kết quả là: đặc trưng thời gian tại vị trí (t, f) chỉ được giữ lại đầy đủ khi đặc trưng tần số tại cùng vị trí đó xác nhận sự quan trọng (giá trị sigmoid gần 1), và bị ức chế khi không được xác nhận (giá trị sigmoid gần 0). Cơ chế đối xứng áp dụng cho chiều ngược lại.

Thiết kế này dựa trên quan sát ngôn ngữ học quan trọng: trong tiếng nói tự nhiên, một âm vị không thể được mô tả đầy đủ bởi chỉ một chiều duy nhất – nguyên âm /a/ có cả cấu trúc formant đặc trưng (chiều tần số) lẫn độ dài thời gian nhất định (chiều thời gian). Cổng chéo mô hình hoá sự phụ thuộc liên tục này một cách tường minh: chỉ những đặc trưng được xác nhận từ *cả hai* chiều mới được giữ lại hoàn toàn, tạo ra biểu diễn liên tục đáng tin cậy hơn.

Bước 4 – Fuse và kết nối tắt: Hai nhánh sau cổng được cộng lại và đưa qua lớp Fuse (tích chập 1×1 + BN + ReLU) để học cách tổng hợp thông tin liên tục. Sau đó, kết nối tắt từ feature map trung gian $\hat{\mathbf{F}}$ (qua tích chập 1×1) được cộng thêm vào:

$$\mathbf{F}_{out} = \text{Fuse}(\mathbf{F}'_{time} + \mathbf{F}'_{freq}) + \text{Conv}_{1 \times 1}(\hat{\mathbf{F}}) \quad (3.14)$$

Kết nối tắt từ $\hat{\mathbf{F}}$ đóng vai trò quan trọng: nó đảm bảo rằng toàn bộ thông tin từ feature map đã giảm mẫu không bị mất hoàn toàn qua cơ chế cổng – ngay cả khi một phần thông tin bị ức chế bởi cổng chéo, kết nối tắt vẫn truyền tải thông tin đó vào đầu ra. Điều này tạo ra sự cân bằng giữa tính chọn lọc (qua cổng) và tính bảo toàn (qua kết nối tắt).

3.3.5 Đầu phân loại (Classification Head)

Sau ba giai đoạn xử lý, feature map $\mathbf{F} \in \mathbb{R}^{C_3 \times 16 \times 16}$ chứa thông tin đặc trưng phong phú về từ khóa đầu vào nhưng vẫn còn chiều không gian dư thừa. Đầu phân loại có nhiệm vụ chất lọc thông tin này thành vector quyết định 35 chiều qua bốn bước:

Global Average Pooling (GAP): Thu gọn toàn bộ chiều không gian 16×16 về một điểm duy nhất trên mỗi kênh bằng cách lấy trung bình:

$$\mathbf{z}_c = \frac{1}{16 \times 16} \sum_{i,j} \mathbf{F}_c(i,j), \quad c = 1, \dots, C_3 \quad (3.15)$$

Tạo ra vector đặc trưng toàn cục $\mathbf{z} \in \mathbb{R}^{C_3}$ đại diện cho toàn bộ spectrogram [18]. GAP được ưu tiên hơn lớp Fully Connected vì hai lý do: giảm mạnh số tham số (từ $C_3 \times H \times W$ đầu vào xuống còn C_3) và tạo ra bất biến với vị trí – từ khóa được nhận diện dựa trên sự hiện diện của đặc trưng, không phụ thuộc vào vị trí cụ thể trong feature map.

Flatten: Trải phẳng tensor $C_3 \times 1 \times 1$ về vector 1D $\mathbf{z} \in \mathbb{R}^{C_3}$.

Dropout: Ngẫu nhiên tắt một tỷ lệ nơ-ron trong quá trình huấn luyện [16]. Dropout tại đầu phân loại đặc biệt cần thiết cho mô hình nhỏ gọn: khi mô hình có ít tham số, nguy cơ mỗi nơ-ron “ghi nhớ” thông tin huấn luyện thay vì học đặc trưng tổng quát là cao hơn. Dropout buộc mỗi nơ-ron phải hữu ích độc lập, không phụ thuộc vào sự hiện diện của các nơ-ron khác.

Linear: Lớp tuyến tính $\mathbf{W} \in \mathbb{R}^{35 \times C_3}$ ánh xạ vector đặc trưng sang không gian 35 lớp từ khóa. Đầu ra là vector logit $\mathbf{z}_{out} \in \mathbb{R}^{35}$ – được đưa qua softmax khi suy luận để tạo phân phối xác suất, hoặc trực tiếp vào hàm mất mát cross-entropy khi huấn luyện.

3.3.6 Phân tích tổng hợp và vị trí trong hệ sinh thái KWS

Ba đặc trưng thiết kế của QANet-Lite

Nhìn lại toàn bộ kiến trúc, QANet-Lite có ba đặc trưng thiết kế nổi bật phân biệt với các kiến trúc KWS nhẹ hiện có:

(1) Phân tích đa trục chuyên biệt: Thay vì dùng bộ lọc vuông $k \times k$ xử lý đồng đều cả hai chiều, QANet-Lite tách biệt phân tích theo trục thời gian (1×3), trục tần số (3×1), không gian cục bộ (3×3 DW-Sep), và kênh (CAM). Sự phân biệt này phù hợp với đặc điểm vật lý của spectrogram: hai trục có ngữ nghĩa hoàn toàn khác nhau và cần được phân tích chuyên biệt.

(2) Định tuyến động thích nghi: Bộ định tuyến trong QuadAxialBlock học cách phân bổ trọng số cho bốn nhánh tùy theo nội dung từng mẫu đầu vào – khả năng thích nghi quan trọng khi phải xử lý 35 lớp từ khóa với đặc trưng âm học đa dạng trong cùng một kiến trúc.

(3) Cổng chéo liên trục: CrossAxisDownsample không chỉ giảm mẫu mà còn lọc thông tin theo cổng chéo – đặc trưng thời gian được xác nhận bởi thông tin tần số và ngược lại. Cơ chế này mô hình hoá tường minh sự phụ thuộc liên trục trong tiếng nói mà pooling truyền thống không thể làm.

Hiệu quả tham số và phù hợp thiết bị biên

Tất cả ba đặc trưng trên đạt được với chi phí tham số tối thiểu. Bộ định tuyến chỉ thêm $4 \times C_{in}$ tham số. CAM thêm $2C^2/r$ tham số (với $r = 4$). Việc dùng DW-SepConv trong nhánh Local và DWConv trong CrossAxisDownsample giữ chi phí tính toán thấp hơn đáng kể so với tích chập tiêu chuẩn. Tổng thể, QANet-Lite đạt được khả năng biểu diễn đa trực phong phú trong ngân sách tham số và FLOPs thấp, phù hợp với ràng buộc của thiết bị biên.

3.4 Chiến lược chưng cất tri thức

3.4.1 Tổng quan luồng huấn luyện

QANet-Lite được huấn luyện theo khung Giáo viên – Học sinh (Teacher-Student framework) [30]: một mô hình Teacher lớn và mạnh (AST với 98.17% accuracy trên tập kiểm tra) hướng dẫn QANet-Lite(Student) học không chỉ từ nhãn cứng (hard labels) của tập dữ liệu mà còn từ phân phối xác suất mềm (soft labels) của Teacher. Teacher bị loại bỏ hoàn toàn sau khi huấn luyện xong; chỉ QANet-Lite được triển khai trên thiết bị biên.

Lý do áp dụng KD trong trường hợp này rõ ràng: Student (CNN nhẹ, vài trăm K tham số) và Teacher (AST Transformer, 85M tham số) có khoảng cách năng lực biểu diễn rất lớn. Huấn luyện Student trực tiếp chỉ từ nhãn one-hot bỏ qua lượng lớn tri thức mà Teacher đã tích lũy – thông tin về mức độ tương đồng giữa các từ khóa, ranh giới quyết định mềm giữa các lớp, và những mẫu nào khó phân biệt. Phân phối xác suất của Teacher truyền tải toàn bộ thông tin này dưới dạng “nhãn mềm” phong phú hơn nhiều so với nhãn one-hot.

3.4.2 Hàm mất mát tổng hợp

Hàm mất mát huấn luyện của QANet-Lite kết hợp hai thành phần bổ sung nhau:

Hàm mất mát cứng (Hard Loss): Cross-Entropy tiêu chuẩn giữa logits của Student

và nhãn one-hot từ tập dữ liệu:

$$\mathcal{L}_{hard} = - \sum_{i=1}^C y_i \log P_{S,i}, \quad P_{S,i} = \text{softmax}(\mathbf{z}_S)_i \quad (3.16)$$

Thành phần này neo Student vào nhãn thực, đảm bảo mô hình không bị “lệch” hoàn toàn theo phân phối của Teacher khi Teacher mắc lỗi.

Hàm mất mát mềm (Soft Loss): Phân kỳ KL giữa phân phối mềm của Teacher P_T^T và Student P_S^T , cả hai được tính ở nhiệt độ $T > 1$:

$$\mathcal{L}_{soft} = T^2 \cdot D_{KL}(P_T^T \parallel P_S^T) \quad (3.17)$$

Hệ số T^2 là yếu tố kỹ thuật quan trọng: do temperature scaling làm co gradient theo tỷ lệ $1/T^2$, hệ số T^2 bù lại để gradient từ $\mathcal{L}_{soft}$ có cùng độ lớn với $\mathcal{L}_{hard}$ bất kể giá trị T được chọn. Điều này đảm bảo tham số α điều chỉnh cân bằng hai thành phần một cách có ý nghĩa và nhất quán.

Hàm mất mát tổng hợp:

$$\mathcal{L} = (1 - \alpha) \cdot \mathcal{L}_{hard} + \alpha \cdot \mathcal{L}_{soft} \quad (3.18)$$

3.4.3 Lựa chọn và lý giải siêu tham số α và T

Nhiệt độ $T = 4$: Ở nhiệt độ mặc định $T = 1$, phân phối softmax của Teacher thường rất “nhọn” – xác suất lớp đúng chiếm gần như toàn bộ và các lớp khác gần bằng 0, không khác nhiều so với nhãn one-hot. Giá trị $T > 1$ làm phẳng phân phối, khuếch đại các xác suất nhỏ và lộ rõ mối quan hệ tương đồng giữa các từ khóa. Trong tập Google Speech Commands V2 với 35 lớp, nhiều từ khóa có phát âm gần nhau (“yes”/“no”, “go”/“no”, “up”/“off”): Teacher đã học được những tương đồng này qua quá trình huấn luyện; phân phối mềm với $T = 4$ truyền tải thông tin quý giá đó cho Student. Giá trị $T = 4$ được chọn theo khuyến nghị trong tài liệu gốc của KD [30] cho bài toán phân loại nhiều lớp – đủ lớn để tiết lộ tri thức tương quan liên lớp nhưng không quá lớn để làm mất tính phân biệt. Tại thời điểm suy luận, nhiệt độ được đặt về $T = 1$ để nhận được phân phối “sắc

nét” nhất cho quyết định phân loại.

Tham số cân bằng $\alpha = 0.5$: Giá trị này cân bằng đều hai nguồn tín hiệu huấn luyện, phản ánh một quyết định thiết kế có chủ ý. Trong bối cảnh cross-architecture distillation (Teacher là Transformer, Student là CNN), khoảng cách kiến trúc lớn có nghĩa là Student có thể gặp khó khăn khi bị áp đảo bởi soft labels của Teacher [33]. Việc duy trì $\alpha = 0.5$ thay vì α lớn hơn (như 0.9 trong thiết lập đồng kiến trúc) đảm bảo Student vẫn học đủ từ nhãn cứng để có nền tảng phân loại vững chắc, trong khi vẫn tiếp nhận được tri thức tương đồng liên lớp từ Teacher. Cả hai siêu tham số được giữ cố định trong toàn bộ quá trình huấn luyện, không áp dụng chiến lược điều chỉnh động.

3.5 Triển khai lên phần cứng

3.5.1 Chuyển đổi sang định dạng ONNX

Sau khi huấn luyện hoàn tất trên môi trường PyTorch, mô hình được chuyển đổi sang định dạng ONNX (Open Neural Network Exchange) để triển khai lên thiết bị biên. ONNX là định dạng biểu diễn mô hình học sâu trung gian, cho phép chạy suy luận hiệu quả trên nhiều nền tảng phần cứng khác nhau thông qua ONNX Runtime.

Quá trình chuyển đổi thực hiện theo hai bước. Đầu tiên, mô hình PyTorch được xuất sang file .onnx bằng hàm `torch.onnx.export()` với đầu vào mẫu kích thước $1 \times 1 \times 128 \times 128$. Tiếp theo, tính đúng đắn của file ONNX được kiểm tra bằng `onnx.checker.check_model()` và kết quả suy luận được đối chiếu với đầu ra gốc của PyTorch để đảm bảo không có sai lệch số học trong quá trình chuyển đổi.

3.5.2 Quy trình đánh giá hiệu năng suy luận (Benchmark Pipeline)

Để đánh giá chính xác mức độ hiệu quả của cấu trúc mạng đề xuất, quy trình thực nghiệm trên thiết bị NVIDIA Jetson Nano (cấu hình MAXN) được thiết lập theo chuẩn benchmark phần cứng. Thay vì triển khai một hệ thống đầu cuối (end-to-end) bao gồm cả độ trễ của micro và trích xuất đặc trưng trực tuyến, quy trình này cô lập mô hình học sâu để đo lường thuần túy năng lực tính toán của vi mạch:

1. **Tiền xử lý (Offline Pre-processing):** Toàn bộ các mẫu âm thanh tĩnh 1 giây từ tập kiểm thử (test set) được chuyển đổi trước thành ảnh 2D Log-Mel Spectrogram kích thước $1 \times 128 \times 128$ và chuẩn hoá theo thống kê (μ, σ) của tập huấn luyện. Bước này loại bỏ biến số về thời gian xử lý tín hiệu số (DSP), giúp tập trung đánh giá tốc độ của mạng CNN.
2. **Khởi tạo và nạp dữ liệu:** Các ảnh phổ 2D được nạp vào bộ nhớ của Jetson Nano dưới dạng tensor với kích thước batch $B = 1$. Việc suy luận từng mẫu đơn lẻ nhằm mô phỏng sát nhất điều kiện xử lý dữ liệu theo thời gian thực (streaming data) trên thiết bị biên.
3. **Suy luận và đo lường (Inference & Profiling):** ONNX Runtime thực thi đồ thị tính toán của mô hình QANet-Lite đối với từng tensor đầu vào. Xuyên suốt quá trình chạy toàn bộ tập kiểm thử, tiện ích tegrastats được kích hoạt chạy ngầm để ghi nhận liên tục độ trễ xử lý (latency) và công suất tiêu thụ điện (power consumption) của SoC.
4. **Phân loại và đối chiếu:** Đầu ra của mạng là vector logit 35 chiều. Nhãn dự đoán cuối cùng được xác định trực tiếp bằng phép toán argmax (lấy chỉ số có giá trị lớn nhất) và đem đối chiếu với nhãn gốc (ground truth) để tổng hợp độ chính xác (Accuracy).

Chương 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường thực nghiệm

4.1.1 Tập dữ liệu

Toàn bộ thực nghiệm được thực hiện trên tập dữ liệu **Google Speech Commands V2** [9] – bộ dữ liệu chuẩn quốc tế được sử dụng rộng rãi nhất trong nghiên cứu KWS hiện đại. Tập dữ liệu bao gồm 105.829 đoạn âm thanh 1 giây thuộc 35 lớp từ khóa, được thu âm ở tần số lấy mẫu 16 kHz bởi hơn 2.000 người dùng trong nhiều điều kiện môi trường khác nhau, tạo ra sự đa dạng tự nhiên về giọng người, ngữ điệu và mức độ nhiễu nền.

Tập dữ liệu được chia theo phân hoạch chuẩn của tác giả: 80% dùng để huấn luyện, 10% để kiểm thử trong quá trình phát triển (validation), và 10% còn lại làm tập đánh giá cuối (test set). Trong quá trình huấn luyện, ba kỹ thuật tăng cường dữ liệu được áp dụng đồng thời:

- **Thêm nhiễu nền:** Chồng ngẫu nhiên các đoạn nhiễu môi trường (tiếng đường phố, văn phòng, quán cà phê) lên tín hiệu gốc với tỉ lệ tín-nhiều (SNR) ngẫu nhiên trong khoảng $[0, 15]$ dB.
- **SpecAugment** [21]: Che ngẫu nhiên tối đa 2 dải thời gian (mỗi dải tối đa 25 khung)

và 2 dải tần số (mỗi dải tối đa 20 bin) trực tiếp trên Log-Mel Spectrogram.

- **Mixup** [22]: Nội suy tuyến tính giữa các cặp mẫu huấn luyện với hệ số $\lambda \sim \text{Beta}(0.5, 0.5)$.

4.1.2 Cấu hình phần cứng

Thực nghiệm được tiến hành trên hai nền tảng phần cứng phục vụ hai mục đích khác nhau, như mô tả trong Bảng 4.1.

Bảng 4.1: Cấu hình phần cứng sử dụng trong thực nghiệm

Thành phần	Máy trạm huấn luyện	Thiết bị biên
Nền tảng	Workstation	NVIDIA Jetson Nano Developer Kit
GPU	NVIDIA RTX 3070 (8 GB)	Maxwell 128-core GPU
CPU	Intel Core i7	ARM Cortex-A57 (4 nhân)
RAM	32 GB	4 GB LPDDR4
Chế độ	–	MAXN (10 W)
Framework	PyTorch 2.x	ONNX Runtime 1.x
Đo điện năng	–	tegrastats

4.1.3 Cấu hình huấn luyện

Quá trình huấn luyện được thực hiện theo hai pha. Pha đầu tiên là huấn luyện mô hình Teacher (AST) riêng lẻ đến khi hội tụ để có nguồn soft labels chất lượng cao. Pha thứ hai là huấn luyện QANet-Lite với hàm mất mát chùng cất tri thức, trong đó trọng số Teacher bị đóng băng hoàn toàn. Các siêu tham số huấn luyện được trình bày trong Bảng 4.2.

Bảng 4.2: Siêu tham số huấn luyện

Siêu tham số	Teacher (AST)	Student (QANet-Lite)
Optimizer	Adam	Adam
Learning rate	1×10^{-4}	1×10^{-3}
LR Scheduler	ReduceLROnPlateau	$\times 0.5$ mỗi 5 epoch sau epoch 10
Batch size	64	64
Epochs	50	100
Weight decay	1×10^{-4}	1×10^{-4}
Temperature T	–	4
Alpha α	–	0.5

4.2 Đánh giá trên máy trạm

4.2.1 So sánh độ chính xác với các mô hình baseline

Bảng 4.3 trình bày kết quả so sánh toàn diện giữa QANet-Lite và các mô hình tham chiếu trên tập kiểm tra Google Speech Commands V2, được đánh giá trên máy trạm RTX 3070.

Bảng 4.3: Độ phức tạp kiến trúc và hiệu năng cơ sở trên máy trạm hiệu năng cao (RTX 3070)

Mô hình	Params (M)↓	FLOPs (G)↓	Storage (MB)↓	Acc. (%)↑	Throughput (IPS)↑
AST (Teacher)	85.40	26.21	325.81	98.17	212.36
ResNet18	11.18	1.14	42.67	94.91	452.14
MobileNetV2	2.25	0.20	8.60	95.65	359.65
QANet-Lite (Scratch)	0.16	0.39	0.64	93.39	325.07
QANet-Lite (KD)	0.16	0.39	0.64	95.62	335.01

Kết quả cho thấy mô hình AST đạt độ chính xác 98.17% nhưng yêu cầu 325.81 MB bộ nhớ, hoàn toàn không khả thi cho thiết bị biên. QANet-Lite (KD) đạt được sự cân bằng tốt hơn đáng kể: kích thước mô hình chỉ bằng một phần nhỏ so với các baseline thông dụng, trong khi độ chính xác vẫn cạnh tranh nhờ tri thức được chưng cất từ AST

Teacher.

4.3 Đánh giá trên phần cứng biên (Jetson Nano)

Đây là phần đánh giá trọng tâm của nghiên cứu, vì mục tiêu cuối cùng là triển khai hệ thống KWS hoạt động hiệu quả trong điều kiện ràng buộc thực tế của thiết bị biên. Tất cả số liệu được thu thập bằng tiện ích `tegrastats` chạy song song với quá trình suy luận để đảm bảo tính chính xác và khả năng tái lập.

4.3.1 Tốc độ xử lý và Độ trễ (Latency & Throughput)

Bảng 4.4 trình bày kết quả đo đạc đầy đủ trên Jetson Nano. Để hệ thống hoạt động thời gian thực, độ trễ suy luận (Latency) phải nhỏ hơn độ dài của đoạn âm thanh đầu vào (1 giây), tức là Hệ số thời gian thực $RTF < 1.0$.

Bảng 4.4: So sánh hiệu năng trên thiết bị NVIDIA Jetson Nano

Model	Acc. (%)↑	Latency (ms)↓	RTF↓	Throughput (IPS)↑	Power (W)↓	Eff. (IPS/W)↑
AST (Teacher)	98.17	243.81	0.244	4.10	8.49	0.48
ResNet18	94.91	40.69	0.041	24.57	6.65	3.69
MobileNetV2	95.65	11.12	0.011	89.95	7.98	11.27
QANet-Lite (Scratch)	93.39	23.34	0.023	42.84	6.31	6.79
QANet-Lite (KD)	95.62	23.84	0.024	41.95	6.28	6.68

Về mặt tốc độ, MobileNetV2 đạt thông lượng (Throughput) cao nhất với 89.95 IPS. QANet-Lite (KD) đạt thông lượng 41.95 IPS, tương đương $RTF = 0.024$. Dù chậm hơn MobileNetV2 do cấu trúc đường ống phân nhánh phức tạp hơn, QANet-Lite vẫn vượt mức yêu cầu thời gian thực rất xa (chỉ cần $RTF < 0.1$ là đủ để dành tài nguyên cho các tác vụ nền khác). Việc thông lượng không quá cao thực chất lại là một lợi thế quan trọng về mặt điện năng, được phân tích ở phần tiếp theo.

4.3.2 Tiêu thụ điện năng và Nghịch lý Power Paradox

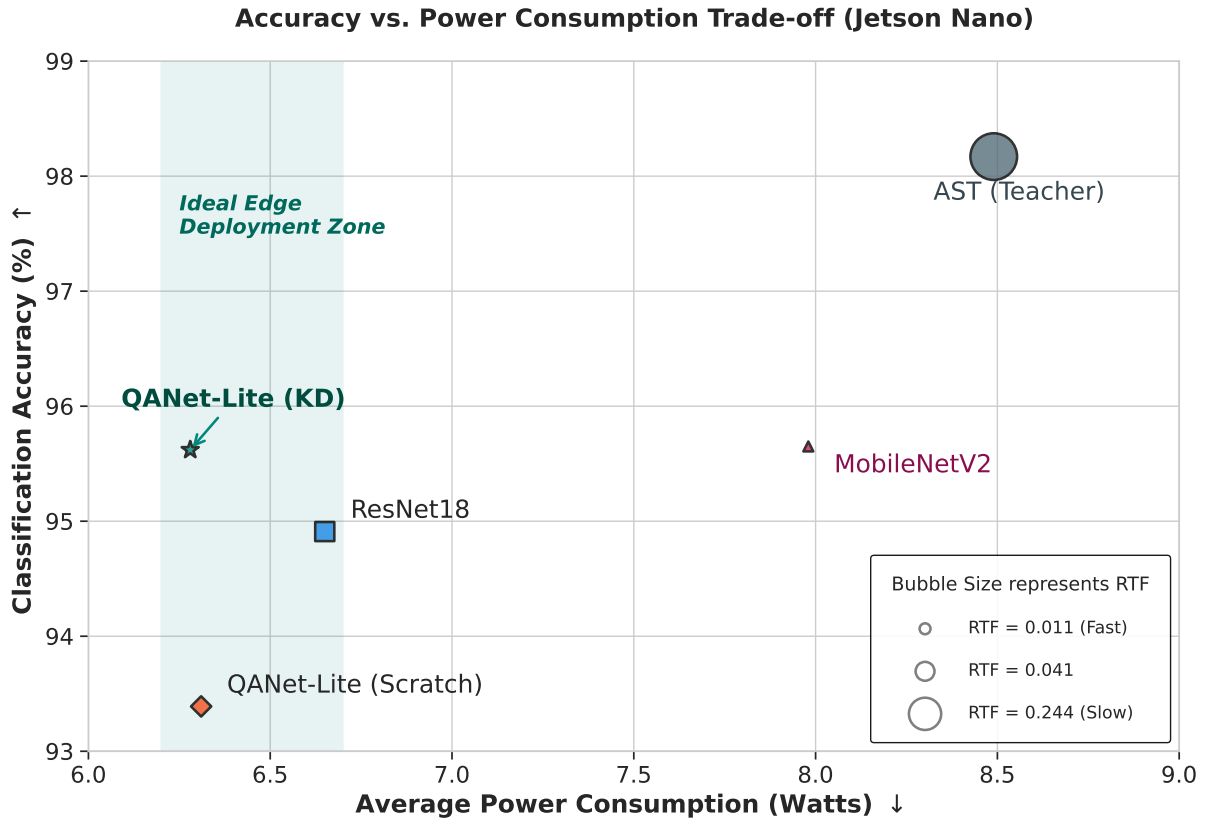
Một phát hiện quan trọng của thực nghiệm là sự tách biệt giữa FLOPs lý thuyết và tiêu thụ điện năng thực tế. Để đánh giá công bằng, năng lượng trung bình mỗi lần suy

luận được tính bằng $E_{inf} = P_{avg}/IPS$ (Joule/inference).

Nhìn vào Bảng 4.4, MobileNetV2 có FLOPs chỉ 0.20 G – thấp nhất trong số các mạng CNN – nhưng lại có công suất tiêu thụ trung bình lên tới 7.98 W. Nguyên nhân là do mô hình này đẩy thông lượng lên quá cao (89.95 IPS), buộc GPU và vi mạch phải hoạt động liên tục ở trạng thái đỉnh, truy xuất bộ nhớ dày đặc mà không có khoảng nghỉ. Điều này cung cấp bằng chứng thực nghiệm rõ ràng cho hiện tượng *Power Paradox* [7, 8].

Ngược lại, QANet-Lite giải quyết nghịch lý này thông qua cấu trúc mạng đa nhánh song song. Việc phân tách luồng dữ liệu tạo ra các khoảng chờ vi mô (pipeline stalls) có chủ đích trong đường ống xử lý. Các khoảng nghỉ này ngăn GPU bị vắt kiệt, giúp duy trì mức công suất tiêu thụ trung bình ở mức thấp nhất (6.28 W). Nhờ đó, năng lượng cho mỗi lần suy luận (E_{inf}) của QANet-Lite (KD) chỉ ở mức 0.150 J/inf, tối ưu hơn đáng kể so với các mô hình nặng như ResNet18 hay Teacher AST.

4.3.3 Sự đánh đổi (Trade-off) lý tưởng cho Thiết bị biên



Hình 4.1: Biểu đồ đánh đổi giữa độ chính xác (Accuracy) và năng lượng tiêu thụ (Power) trên Jetson Nano. Kích thước bong bóng tỷ lệ thuận với Hệ số thời gian thực (RTF).

Hình 4.1 trực quan hoá sự đánh đổi giữa độ chính xác và tiêu thụ điện. Vùng lý tưởng cho triển khai thiết bị biên nằm ở góc trên cùng bên trái (độ chính xác cao, tiêu thụ điện thấp). Biểu đồ cho thấy QANet-Lite (KD) nằm trọn trong vùng lý tưởng này: tiệm cận độ chính xác của MobileNetV2 nhưng lại tiết kiệm được gần 1.7 W công suất, minh chứng cho sự tối ưu của thiết kế.

4.4 Ablation Study

Để định lượng đóng góp riêng lẻ của từng thành phần trong kiến trúc đề xuất, một loạt thực nghiệm ablation được tiến hành theo phương pháp tích lũy tiến (progressive): bắt đầu từ mô hình nền tảng rồi lần lượt thêm từng thành phần. Tất cả các biến thể sử dụng cùng bộ siêu tham số huấn luyện để đảm bảo so sánh công bằng.

Bảng 4.5: Kết quả Ablation Study – đóng góp tích lũy của từng thành phần

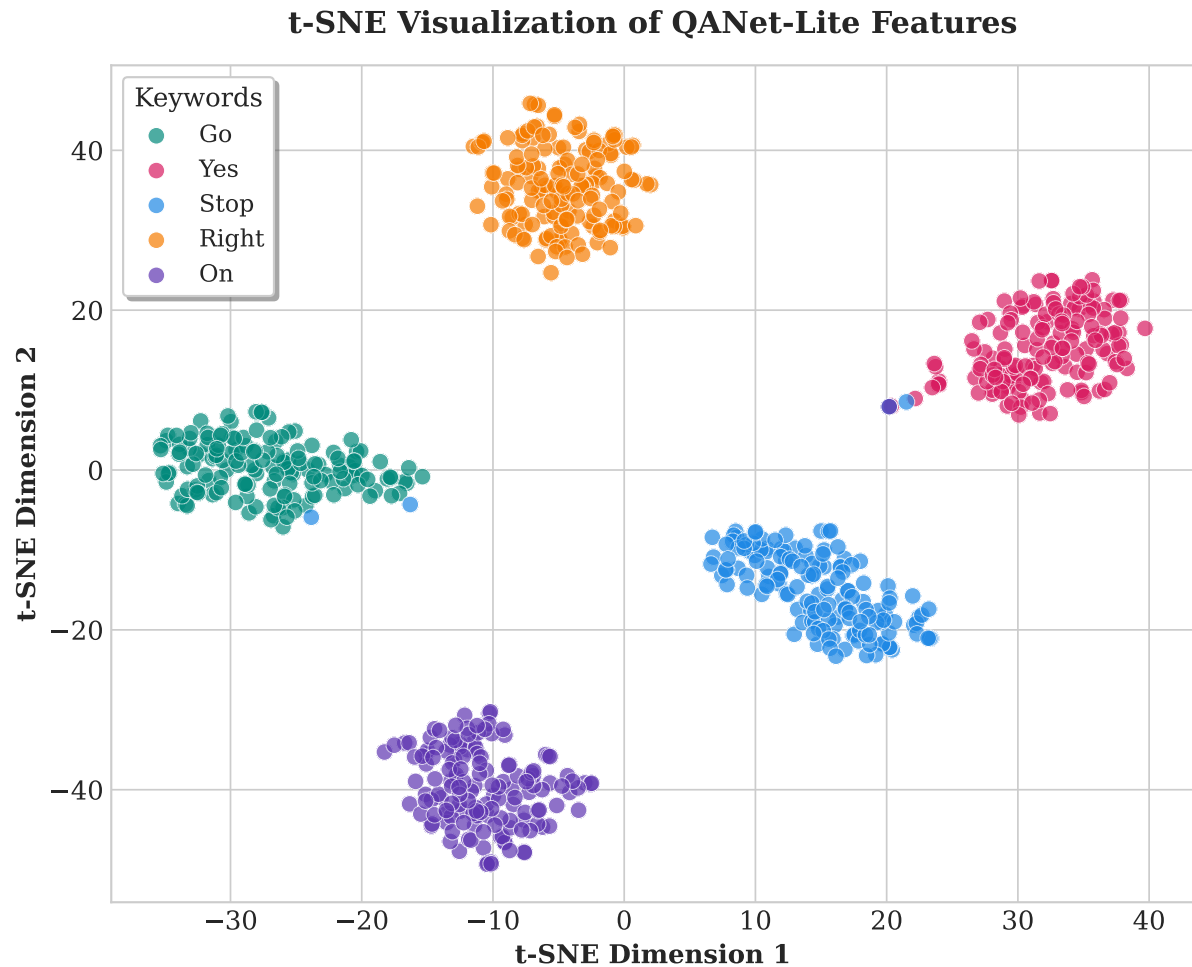
Cấu hình mô hình	CAM	KD	AUC↑	Accuracy (%)↑
QANet-Lite (baseline)	×	×	0.990	93.00
QANet-Lite (+CAM)	✓	×	0.990	93.39
QANet-Lite (Proposed)	✓	✓	0.996	95.62

Đóng góp của cơ chế chú ý (CAM): Mức tăng độ chính xác khi thêm CAM phản ánh khả năng của cơ chế chú ý kênh trong việc ức chế các đặc trưng bị kích hoạt bởi nhiễu nền. Điều này xác nhận quyết định thiết kế nhánh kênh trong QuadAxialBlock không phải là tích chập 1×1 đơn giản mà là một bộ lọc tái cân bằng động (CAM-guided projection), giúp bốn nhánh học được đặc trưng phân biệt hơn.

Đóng góp của Chứng cất tri thức (KD): Sự nhảy vọt về độ chính xác (tăng 2.23%) và AUC khi áp dụng KD minh chứng cho giá trị của các nhãn mềm từ Teacher AST. Bằng cách tiếp thu ranh giới phân loại tinh vi từ mô hình lớn, Student có thể khai thác tối đa cấu trúc phân nhánh để đạt độ chính xác 95.62% mà không làm tăng thêm chi phí suy luận phần cứng.

4.5 Phân tích định tính

4.5.1 Phân tích phân cụm đặc trưng (t-SNE)



Hình 4.2: Biểu đồ t-SNE trực quan hóa không gian đặc trưng của mô hình QANet-Lite. Các cụm từ khóa (keywords) được phân tách rõ ràng, thể hiện khả năng trích xuất đặc trưng mang tính phân loại cao của mô hình.

Để đánh giá chất lượng biểu diễn đặc trưng tại lớp penultimate (trước lớp phân loại tuyến tính), phương pháp t-SNE [34] được áp dụng để chiếu đặc trưng 128 chiều xuống không gian 2D. Hình 4.2 trình bày kết quả trực quan hoá cho 5 từ khóa đại diện trên tập kiểm tra.

Kết quả t-SNE cho thấy QANet-Lite tạo ra các cụm phân tách rất rõ ràng với biên giới liên lớp (inter-class margin) rành mạch. Các mẫu (samples) thuộc cùng một lớp từ khóa quần tụ chặt chẽ với nhau, trong khi các cụm từ khóa khác nhau được đẩy ra xa. Điều

này minh chứng cho khả năng trích xuất đặc trưng mang tính phân biệt sắc bén của kiến trúc đa nhánh đề xuất. Đồng thời, không gian đặc trưng có cấu trúc tốt này cũng phản ánh tính hiệu quả của phương pháp chưng cất tri thức (KD): thông qua soft labels của Teacher [26], mô hình Student không chỉ học cách phân loại cứng nhắc mà còn nắm bắt được mức độ tương đồng ngữ âm thực tế giữa các từ khóa, từ đó hình thành các ranh giới quyết định tự nhiên và có khả năng tổng quát hóa cao.

4.6 Thảo luận kết quả

4.6.1 Phân tích điểm mạnh và hạn chế

Điểm mạnh: QANet-Lite đạt được sự cân bằng vượt trội. Thứ nhất, thiết kế QuadAxialBlock khai thác hiệu quả tính anisotropic của spectrogram thông qua bốn nhánh độc lập; cơ chế Router học động giúp mô hình tự điều chỉnh mức đóng góp tùy theo mẫu đầu vào. Thứ hai, sự tách bạch chức năng mang lại hiệu quả kép: *cấu trúc phân nhánh* tạo khoảng nghỉ vi mô giúp giảm tiêu thụ điện (vượt qua Power Paradox), trong khi *cơ chế CAM* ức chế nhiễu để duy trì độ chính xác. Thứ ba, Knowledge Distillation từ AST [26] thu hẹp đáng kể khoảng cách hiệu năng với các mô hình Transformer mà không làm tăng chi phí tính toán thực tế [30].

Hạn chế: Một số hạn chế cần được thừa nhận. Đầu tiên, việc phụ thuộc vào Teacher AST trong pha huấn luyện làm phức tạp pipeline và đòi hỏi GPU có VRAM đủ lớn để chạy song song cả hai mô hình. Thứ hai, kiến trúc CNN với trường thụ cảm hữu hạn vẫn gặp khó khăn trong việc mô hình hoá các phụ thuộc dài hạn (long-range dependencies) trong quang phổ âm thanh, vốn là điểm mạnh của Transformer. Thứ ba, mô hình hiện tại chưa được đánh giá trên các ngôn ngữ khác tiếng Anh hay trong điều kiện nhiễu cực đoan, giới hạn khả năng tổng quát hoá.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Nghiên cứu này trình bày thiết kế và đánh giá toàn diện QANet-Lite – một kiến trúc mạng nơ-ron tích chập siêu nhẹ được đề xuất cho bài toán Nhận dạng từ khóa (KWS) trên thiết bị biên. Xuất phát từ ba thách thức cốt lõi của Edge AI (giới hạn bộ nhớ, ràng buộc điện năng và nghịch lý Power Paradox), nghiên cứu đề xuất một hướng tiếp cận tổng thể kết hợp thiết kế kiến trúc chuyên biệt, cơ chế chú ý kênh và chứng cất tri thức liên kiến trúc.

Các đóng góp chính của nghiên cứu có thể được tóm tắt như sau:

Thứ nhất – Kiến trúc QuadAxialBlock: Nghiên cứu đề xuất khối trích xuất đặc trưng bốn nhánh (QuadAxialBlock) khai thác tính anisotropic của Log-Mel Spectrogram bằng cách phân rã bài toán học đặc trưng thành bốn luồng song song chuyên biệt: nhánh cục bộ (Local, DW-Sep Conv 3×3), nhánh thời gian (Time, Conv 1×3), nhánh tần số (Freq, Conv 3×1) và nhánh kênh (Channel, CAM-guided). Cơ chế Router với trọng số w_1-w_4 được học động giúp mô hình tự điều chỉnh mức độ đóng góp của từng nhánh tùy theo đặc tính của từng mẫu đầu vào. Thiết kế này cho phép mô hình nắm bắt đặc trưng âm học đa chiều mà không cần tăng kích thước kernel hay độ sâu mạng, từ đó kiểm soát

được số tham số trong giới hạn cho phép của thiết bị nhúng.

Thứ hai – Phân tách chức năng tối ưu phần cứng và độ chính xác: Thiết kế đề xuất đạt được hiệu quả kép thông qua sự phân bạch rõ ràng về chức năng. Trong khi cơ chế chú ý kênh (CAM) làm nhiệm vụ ức chế đặc trưng nhiễu nền để duy trì độ chính xác cao, thì bản thân cấu trúc đường ống phân nhánh của QuadAxialBlock chủ động tạo ra các khoảng chờ vi mô (pipeline stalls) giúp điều hòa nhịp độ tính toán. Sự điều tiết này ngăn chặn SoC bị vắt kiệt bởi các luồng truy cập bộ nhớ liên tục, từ đó làm giảm đáng kể mức tiêu thụ điện năng trung bình so với các mô hình CNN tối ưu FLOPs thuần túy.

Thứ ba – Chứng cất tri thức liên kiến trúc: Chiến lược KD từ AST Teacher (85M tham số) sang QANet-Lite cải thiện độ chính xác mà hoàn toàn không làm tăng chi phí suy luận sau triển khai. Kết quả thực nghiệm xác nhận rằng CAM đóng vai trò bộ dịch đặc trưng (feature translator) hiệu quả, giúp thu hẹp khoảng cách năng lực biểu diễn giữa Transformer Teacher và CNN Student.

Thứ tư – Xác nhận thực nghiệm Power Paradox: Thực nghiệm trên NVIDIA Jetson Nano cung cấp bằng chứng định lượng cho hiện tượng Power Paradox: mô hình có FLOPs thấp nhất (MobileNetV2, 0.20 G) lại tiêu thụ điện cao nhất (7.98 W) do throughput quá cao buộc SoC hoạt động liên tục ở trạng thái đỉnh. Đây là cảnh báo quan trọng cho cộng đồng nghiên cứu về việc sử dụng FLOPs làm chỉ số duy nhất đánh giá hiệu quả mô hình cho ứng dụng biên.

Nhìn tổng thể, QANet-Lite đạt được sự cân bằng vượt trội giữa bốn tiêu chí: độ chính xác phân loại, kích thước mô hình, độ trễ suy luận và tiêu thụ điện năng – xác nhận tính khả thi của hướng tiếp cận kết hợp thiết kế kiến trúc chuyên biệt với chứng cất tri thức cho bài toán KWS trên thiết bị biên trong thực tế.

5.2 Hạn chế của nghiên cứu

Mặc dù đạt được các kết quả đề ra, nghiên cứu này vẫn tồn tại một số hạn chế cần được thừa nhận minh bạch:

Chưa triển khai ứng dụng đầu cuối (End-to-End): Hệ thống hiện tại được đánh giá theo chuẩn benchmark phần cứng tĩnh (offline pre-processing) nhằm đo lường thuần

túy năng lực tính toán của vi mạch. Nghiên cứu chưa tích hợp mô hình vào một luồng ứng dụng thực tế bao gồm quá trình thu âm liên tục từ micro, tiền xử lý tín hiệu số (DSP) và trích xuất đặc trưng trực tuyến (streaming audio processing).

Phụ thuộc vào Teacher offline: Quy trình huấn luyện hiện tại yêu cầu chạy song song cả AST Teacher (325 MB) và QANet-Lite trên cùng một GPU, đòi hỏi VRAM đủ lớn và tăng thời gian huấn luyện. Điều này có thể là rào cản cho các nhóm nghiên cứu với tài nguyên tính toán hạn chế.

Giới hạn phụ thuộc dài hạn: Kiến trúc CNN với trường thụ cảm hữu hạn gặp khó khăn trong việc mô hình hoá các phụ thuộc dài hạn (long-range dependencies) trong quang phổ âm thanh. Với các từ khóa dài hoặc có cấu trúc phức tạp, hạn chế này có thể ảnh hưởng đến độ chính xác.

Phạm vi đánh giá phần cứng: Thử nghiệm phần cứng chỉ được tiến hành trên NVIDIA Jetson Nano – một nền tảng ARM Cortex-A57 với GPU Maxwell tích hợp. Kết quả có thể không tổng quát hoàn toàn sang các nền tảng khác như Raspberry Pi, STM32 hay các vi điều khiển ARM Cortex-M với kiến trúc phần cứng và bộ tối ưu phần mềm khác nhau.

Chưa áp dụng lượng tử hoá: Nghiên cứu chưa khảo sát tác động của quantization (int8 hay int4) lên cả độ chính xác lẫn hiệu năng phần cứng. Đây là bước tối ưu hoá quan trọng thường được áp dụng trong các hệ thống TinyML triển khai thực tế.

5.3 Hướng phát triển

Dựa trên kết quả và hạn chế của nghiên cứu hiện tại, một số hướng phát triển tiếp theo được đề xuất:

Phát triển hệ thống End-to-End trong thực tế: Xây dựng một ứng dụng hoàn chỉnh tích hợp mô hình vào luồng xử lý âm thanh trực tiếp (streaming audio). Hướng đi này tập trung vào việc xử lý tín hiệu từ micro, trích xuất ảnh phổ theo thời gian thực và áp dụng thuật toán cửa sổ trượt (sliding window) để ra quyết định. Mục tiêu cốt lõi là nhúng toàn bộ pipeline này vào một thiết bị vật lý cụ thể (như cụm điều khiển nhà thông minh), hiện thực hóa việc đưa AI ứng dụng trực tiếp vào phần cứng thao tác hằng ngày.

Tự chưng cất (Self-Distillation): Để loại bỏ sự phụ thuộc vào Teacher AST ngoại lai và đơn giản hoá pipeline huấn luyện, hướng nghiên cứu tiếp theo có thể khám phá các phương pháp self-distillation – trong đó mô hình học từ chính các phiên bản sâu hơn của mình trong quá trình huấn luyện – hoặc online distillation không yêu cầu Teacher được huấn luyện trước [35].

Tích hợp State Space Models (SSM): Các mô hình không gian trạng thái tuyến tính (Linear State Space Models) như Mamba [4] cung cấp khả năng mô hình hoá chuỗi với độ phức tạp $\mathcal{O}(N)$ thay vì $\mathcal{O}(N^2)$ của self-attention. Việc thay thế hoặc kết hợp các khối QuadAxialBlock với SSM nhỏ gọn có thể mở rộng trường thụ cảm của mô hình để nắm bắt phụ thuộc dài hạn trong âm thanh mà vẫn giữ chi phí tính toán phù hợp với thiết bị biên.

Lượng tử hoá và nén mô hình: Áp dụng Post-Training Quantization (PTQ) hoặc Quantization-Aware Training (QAT) để chuyển đổi mô hình từ Float32 sang int8 có thể giảm thêm kích thước mô hình 4 lần và tăng tốc suy luận trên các phần cứng hỗ trợ SIMD integer operations, đặc biệt phù hợp cho các vi điều khiển ARM Cortex-M không có đơn vị tính toán dấu phẩy động.

Mở rộng sang các ngôn ngữ và miền âm thanh khác: Đánh giá khả năng tổng quát hoá của kiến trúc trên các ngôn ngữ ngoài tiếng Anh (đặc biệt là tiếng Việt với đặc tính ngôn điệu – tonal language) và các tập dữ liệu môi trường thực tế nhiều nhiễu hơn sẽ là bước quan trọng để xác nhận tính ứng dụng rộng rãi của mô hình.

Tài liệu tham khảo

- [1] N. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, and F. Kawsar, “An early resource characterization of deep learning on wearables, smartphones and internet-of-things devices,” in *Proceedings of the International Workshop on Internet of Things towards Applications (IoT-App)*, 2015, pp. 7–12.
- [2] M. Shafique, T. Theocharides, C. Bouganis *et al.*, “An overview of next-generation architectures for machine learning: Roadmap, opportunities and challenges in the iot era,” in *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018, pp. 827–832.
- [3] Y. Zhang, N. Suda, L. Lai, and V. Chandra, “Hello edge: Keyword spotting on microcontrollers,” *arXiv preprint arXiv:1711.07128*, 2018. [Online]. Available: <https://arxiv.org/abs/1711.07128>
- [4] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2024. [Online]. Available: <https://arxiv.org/abs/2312.00752>
- [5] C. Banbury, C. Zhou, I. Fedorov *et al.*, “Micronets: Neural network architectures for deploying tinymml applications on commodity microcontrollers,” *arXiv preprint arXiv:2010.11267*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.11267>
- [6] S. Garai and S. Samui, “Exploring tinymml frameworks for small-footprint keyword spotting: A concise overview,” in *Proceedings of the International Conference on Signal Processing and Communications (SPCOM)*, 2024.
- [7] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, “The efficiency misnomer,” *arXiv preprint arXiv:2110.12894*, 2022. [Online]. Available: <https://arxiv.org/abs/2110.12894>

[//arxiv.org/abs/2110.12894](https://arxiv.org/abs/2110.12894)

- [8] J. Chen, S. Kao, H. He *et al.*, “Run, don’t walk: Chasing higher flops for faster neural networks,” *arXiv preprint arXiv:2303.03667*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.03667>
- [9] P. Warden, “Speech commands: A dataset for limited-vocabulary speech recognition,” *arXiv preprint arXiv:1804.03209*, 2018. [Online]. Available: <https://arxiv.org/abs/1804.03209>
- [10] Y. Bengio, “Learning deep architectures for ai,” *Foundations and Trends in Machine Learning*, vol. 2, no. 1, pp. 1–127, 2009. [Online]. Available: <https://arxiv.org/abs/1206.5538>
- [11] A. Howard, M. Zhu, B. Chen *et al.*, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [12] A. Howard, M. Sandler, G. Chu *et al.*, “Searching for mobilenetv3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324. [Online]. Available: <https://arxiv.org/abs/1905.02244>
- [13] D. Rumelhart, G. Hinton, and R. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [14] D. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [15] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2015, pp. 448–456. [Online]. Available: <https://arxiv.org/abs/1502.03167>
- [16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research (JMLR)*, vol. 15, no. 56, pp. 1929–1958, 2014.

- [17] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [18] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014. [Online]. Available: <https://arxiv.org/abs/1312.4400>
- [19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [20] C. Szegedy, W. Liu, Y. Jia *et al.*, “Going deeper with convolutions,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9. [Online]. Available: <https://arxiv.org/abs/1409.4842>
- [21] D. Park, W. Chan, Y. Zhang *et al.*, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Proceedings of Interspeech*, 2019, pp. 2613–2617.
- [22] H. Zhang, M. Cisse, Y. Dauphin, and D. Lopez-Paz, “Mixup: Beyond empirical risk minimization,” *arXiv preprint arXiv:1710.09412*, 2018. [Online]. Available: <https://arxiv.org/abs/1710.09412>
- [23] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [24] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.
- [25] S. Woo, J. Park, J. Lee, and I. Kweon, “Cbam: Convolutional block attention module,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.
- [26] Y. Gong, Y. Chung, and J. Glass, “Ast: Audio spectrogram transformer,” in *Proceedings of Interspeech*, 2021, pp. 571–575.

- [27] S. Majumdar and B. Ginsburg, “Matchboxnet: 1d time-channel separable convolutional neural network architecture for speech commands recognition,” in *Proceedings of Interspeech*, 2020, pp. 3356–3360.
- [28] S. Choi, S. Seo, B. Shin *et al.*, “Temporal convolution for real-time keyword spotting on mobile devices,” in *Proceedings of Interspeech*, 2019.
- [29] A. Berg, M. O’Connor, and M. Cruz, “Keyword transformer: A self-attention model for keyword spotting,” in *Proceedings of Interspeech*, 2021, pp. 4249–4253.
- [30] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [31] S. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghasemzadeh, “Improved knowledge distillation via teacher assistant,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 5191–5198. [Online]. Available: <https://arxiv.org/abs/1902.03393>
- [32] D. Hand and R. Till, “A simple generalisation of the area under the roc curve for multiple class classification problems,” *Machine Learning*, vol. 45, no. 2, pp. 171–186, 2001. [Online]. Available: <https://doi.org/10.1023/A:1010920819831>
- [33] A. Romero, N. Ballas, S. Kahou *et al.*, “Fitnets: Hints for thin deep nets,” *arXiv preprint arXiv:1412.6550*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6550>
- [34] L. van der Maaten and G. Hinton, “Visualizing data using t-sne,” *Journal of Machine Learning Research (JMLR)*, vol. 9, no. 86, pp. 2579–2605, 2008.
- [35] Y. Zhang, T. Xiang, T. Hospedales, and H. Lu, “Deep mutual learning,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4320–4328. [Online]. Available: <https://arxiv.org/abs/1706.00384>

Vo Huan

KHOÁ LUẬN TỐT NGHIỆP.pdf

 aa daad Ho Chi Minh City University of Technology and Engineering (HCMUTE)

Thông tin chi tiết về tài liệu

ID bài nộp

trn:oid:::1:3600862397

Ngày nộp

19:34 GMT+7 24 thg 6, 2026

Ngày tải xuống

19:43 GMT+7 24 thg 6, 2026

Tên tệp

KHOÁ_LUẬN_TỐT_NGHIỆP.pdf

Kích cỡ tệp

4.5 MB

84 Trang

22.641 Từ

91.166 Ký tự

3% Tính tương đồng nói chung

Tổng cộng của tất cả các kết quả trùng khớp, bao gồm cả các nguồn trùng lặp, cho mỗi c...

Đã lọc khỏi Báo cáo

- Mục lục tham khảo
- Văn bản được trích dẫn

Nguồn hàng đầu

- 3%  Nguồn Internet
- 2%  Ấn bản
- 0%  Bài tập được nộp (bài của học sinh)

Cờ chỉ báo về tính toàn vẹn

0 cờ chỉ báo về tính toàn vẹn để đánh giá

Không phát hiện thấy văn bản nào nghi ngờ là ngụy tạo.

Các thuật toán trong hệ thống của chúng tôi xem xét kỹ lưỡng một tài liệu để phát hiện sự không nhất quán khiến cho tài liệu đó khác với một bài nộp bình thường. Nếu nhận thấy điều gì lạ, chúng tôi sẽ gắn cờ để bạn xem xét.

Cờ không nhất thiết là dấu hiệu chỉ báo có vấn đề. Tuy nhiên, bạn nên chú ý vào phần đó để xem xét thêm.

Nguồn hàng đầu

- 3%  Nguồn Internet
- 2%  Ấn bản
- 0%  Bài tập được nộp (bài của học sinh)

Nguồn hàng đầu

Các nguồn có số lượng kết quả trùng khớp cao nhất trong bài nộp. Các nguồn chõng chéo sẽ không được hiển thị.

1	Internet	
ictu.edu.vn		2%
2	Internet	
www.ctu.edu.vn		<1%